vitobotta documentation

Generated on: 13/01/2026

Total pages crawled: 20

Source: https://vitobotta.github.io/hetzner-k3s/

Table of Contents

Section:

https://vitobotta.github.io/hetzner-k3s/

hetzner-k3s

The easiest and fastest way to create
production-ready Kubernetes clusters on Hetzner Cloud

❤️ Support This Project

hetzner-k3s is maintained by a single developer. If it saves you time or money, please consider sponsoring its continued development.

Become a sponsor →

What is hetzner-k3s?

hetzner-k3s is a CLI tool that creates fully-configured Kubernetes clusters on Hetzner Cloud in minutes. It uses k3s, a lightweight Kubernetes distribution by Rancher, and automatically configures everything you need for production workloads.

Key Highlights

Metric	Value
Time to create a 6-node HA cluster	2-3 minutes
Tested scale	500 nodes in under 11 minutes

Metric	Value
Dependencies	Just the CLI tool
Platform fees	None — you only pay Hetzner

What Gets Installed Automatically

- **k3s** — lightweight, certified Kubernetes
- **Hetzner Cloud Controller Manager** — automatic load balancer provisioning
- **Hetzner CSI Driver** — persistent volumes via Hetzner block storage
- **System Upgrade Controller** — zero-downtime k3s upgrades
- **Cluster Autoscaler** — automatic node scaling based on demand
- **Private networking and firewall** — secure cluster communication

---

Getting Started

Installation	Install hetzner-k3s on macOS, Linux, or Windows (via WSL)
Create Your First Cluster	Configuration reference and cluster creation
Complete Tutorial	Set up a cluster with ingress, TLS, and a sample application
Why hetzner-k3s?	Compare to managed services and Terraform-based alternatives

---

Why Choose hetzner-k3s?

Speed Without Shortcuts

A 3-master, 3-worker highly available cluster takes just 2-3 minutes to create. This includes provisioning all infrastructure (instances, load balancer, private network, firewall) and deploying k3s with all essential components.

In stress testing, a 500-node cluster (3 masters, 497 workers) was created in under 11 minutes.

Simplicity That Scales

- **No Terraform or Packer** — a single CLI tool handles everything
- **No management cluster** — unlike Cluster API or Claudie, you don't need Kubernetes to create Kubernetes
- **Simple YAML configuration** — human-readable and version-controllable
- **Idempotent operations** — run `create` multiple times safely; it picks up where it left off

Complete Control

- **Your credentials stay local** — the Hetzner API token never leaves your machine
- **No third-party access** — unlike managed services, no external party can access your clusters
- **Open source (MIT License)** — inspect, modify, and contribute to the code
- **No recurring fees** — you only pay Hetzner for infrastructure

Production-Ready Defaults

- **High availability** — distribute masters and workers across locations
- **Autoscaling** — scale worker pools based on resource demands
- **Private networking** — cluster traffic stays off the public internet
- **Automatic upgrades** — the System Upgrade Controller handles rolling updates

---

Documentation Structure

Getting Started

- Installation — Install hetzner-k3s on your system
- Creating a Cluster — Configuration reference and cluster creation
- Setting Up a Complete Stack — Ingress, TLS, and application deployment

Operations

- Cluster Maintenance — Adding nodes, upgrades, and scaling
- Load Balancers — Configuring Hetzner load balancers
- Storage — Persistent volumes and storage options
- Deleting a Cluster — Clean removal of cluster resources

Advanced Topics

- Recommendations — Best practices for different cluster sizes
- Large Clusters (100+ nodes) — Configuration for large-scale deployments
- Private Clusters — Clusters without public IPs
- Masters in Different Locations — Regional high availability
- Floating IP Egress — Consistent outbound IP addresses

Reference

- Comparison with Other Tools — How hetzner-k3s compares to alternatives
- Troubleshooting — Common issues and solutions
- Upgrading from v1.x to v2.x — Migration guide
- Important Upgrade Notes — Version-specific considerations

Community

- Contributing and Support — How to contribute and get help

Why Hetzner Cloud?

Hetzner Cloud offers exceptional value for Kubernetes workloads:

- **Up to 80% lower costs** than AWS, Google Cloud, and Azure
- **Transparent, all-inclusive pricing** — traffic, IPv4/IPv6, DDoS protection, and firewalls included
- **Six global locations** — Germany (Nuremberg, Falkenstein), Finland (Helsinki), USA (Ashburn, Hillsboro), Singapore
- **Flexible instance types** — x86 and ARM architectures, including cost-effective ARM instances (CAX) for budget-friendly clusters

- **25+ years of reliability** — proven infrastructure trusted by companies worldwide

---

About the Author

I'm Vito Botta, Lead Platform Architect at Brella, an event management platform based in Finland. I handle infrastructure, coding, and supporting the development team.

I also spend time as a bug bounty hunter, finding and responsibly reporting security vulnerabilities.

Connect with me at vitobotta.com. I'm available for consultancies around hetzner-k3s and Kubernetes on Hetzner.

Why Sponsor?

This project is maintained by a single developer in my spare time. Sponsorship helps me:

- Respond to issues faster
- Ship new features regularly
- Keep the project compatible with new Hetzner Cloud updates

If hetzner-k3s saves you time or money, please consider supporting its development.

Platinum Sponsors

A huge thank you to Alamos GmbH for sponsoring the development of awesome features!

Backers

Also thanks to @deubert-it, @jonasbadstuebner, @ricristian , @QuentinFAIDIDE for their support!

Code of Conduct

Everyone interacting in the hetzner-k3s project's codebases, issue trackers, chat rooms and mailing lists is expected to follow the code of conduct.

License

Section:

https://vitobotta.github.io/hetzner-k3s/Comparison_with_other_tools/

Why hetzner-k3s Stands Out

There are several ways to run Kubernetes on Hetzner Cloud. This page explains why hetzner-k3s might be the right choice for your needs, with an honest look at how it compares to alternatives.

At a Glance

Factor	hetzner-k3s	Managed Services	Terraform-based	Cluster API
Setup time	2-3 minutes	5-10 minutes	15-30+ minutes	20+ minutes
Dependencies	CLI only	Account signup	Terraform, Packer, HCL	Managemer cluster
Data privacy	Full control	Third-party access	Full control	Full control
Monthly cost	Infrastructure only	Infrastructure + platform fees (scale with cluster size)	Infrastructure only	Infrastructu only
Credential exposure	None	API tokens to third party	None	None
Learning curve	Low	Low	Medium-High	High
Best for	Most Hetzner users	Zero-ops teams	Terraform-native teams	Multi-cloud standardiza

---

What hetzner-k3s Offers

Speed

Creating a highly available cluster with 3 masters and 3 workers takes **2-3 minutes**. This includes:

- Provisioning all infrastructure (instances, load balancer, private network, firewall)
- Deploying k3s to all nodes
- Installing Cloud Controller Manager, CSI driver, System Upgrade Controller, and Cluster Autoscaler

In stress testing, a 500-node cluster was created in under 11 minutes.

Minimal Dependencies

You need exactly three things:

1. A Hetzner Cloud API token
2. An SSH key pair
3. The hetzner-k3s CLI tool

No Terraform, Packer, Ansible, or existing Kubernetes cluster. No need to learn HCL or manage Terraform state.

Lightweight Kubernetes

k3s uses significantly less memory and CPU than standard Kubernetes, leaving more resources for your workloads. It's a single binary, making deployments and upgrades fast and reliable.

Full Data Sovereignty

- Run the tool on your own machine
- Your Hetzner API token never leaves your system
- No third party gains access to your clusters, credentials, or data
- Complete control with no intermediary

Cost Efficiency

- The tool is free and open source

- You only pay for Hetzner infrastructure
- No per-user fees, per-cluster fees, or management fees
- No surprise charges as your team or infrastructure grows

How Alternatives Compare

Managed Services (Cloudfleet, Edka, Syself)

Managed services handle cluster operations for you, which is valuable if you want zero operational overhead.

Considerations

Credential sharing: You provide your Hetzner API token to the service. The provider has ongoing access to your cloud account and can see your workloads. For regulated industries or security-conscious teams, this may be a concern.

Pricing structure: Managed services typically charge: - A base cluster management fee - Per-vCPU fees for worker nodes - Sometimes per-user fees for enterprise tiers

These platform fees add up quickly as clusters grow. A small development cluster might have modest fees, but production clusters with dozens of nodes can see platform costs that rival or exceed the underlying Hetzner infrastructure costs.

For example, Cloudfleet's free tier limits you to 24 vCPUs with standard availability. Beyond that, costs scale with each additional vCPU. The difference between hetzner-k3s (infrastructure only) and managed services becomes substantial at scale—potentially saving hundreds or thousands of euros per month on larger deployments.

Vendor dependency: Your cluster management depends on the service's availability. If the provider changes pricing, terms, or discontinues service, you're affected. Migration requires effort.

When managed services make sense: Teams that prioritize zero operational overhead over cost or data privacy. Organizations where someone else handling infrastructure is worth the ongoing fees.

Terraform-based Solutions (terraform-hcloud-kube-hetzner, etc.)

Terraform-based solutions are powerful and flexible, especially for teams already using Terraform.

Considerations

Multiple dependencies: You need: - Terraform or OpenTofu - Packer (for custom images) - kubectl CLI - hcloud CLI

Each tool requires installation, configuration, and learning.

Learning curve: You need to understand: - Terraform state management - HCL syntax - How changes propagate through Terraform plans - Debugging across multiple tools

Ongoing maintenance: Terraform state must be stored and managed carefully. Upgrades require understanding how Terraform handles infrastructure drift.

What you gain: More flexibility, infrastructure-as-code patterns familiar to platform teams, integration with existing Terraform workflows.

When Terraform-based solutions make sense: Teams already using Terraform extensively. Organizations with platform engineering teams comfortable with IaC tooling.

Claudie

Claudie is designed for multi-cloud and hybrid deployments. It provisions clusters across different cloud providers from a single management interface.

Considerations

Requires a management cluster: Claudie runs on an existing Kubernetes cluster. You need Kubernetes to create Kubernetes.

For production use, this management cluster needs to be resilient since Claudie maintains state for all clusters it provisions.

Significant dependencies: The management cluster needs: - cert-manager - Multiple Claudie components (ansible, builder, claudie-operator, dynamodb, kube-eleven, kuber, minio, mongodb, scheduler, terraformer)

Operational overhead: You're responsible for keeping the management cluster healthy. If it has issues, you can't manage your provisioned clusters.

When Claudie makes sense: Organizations running clusters across multiple cloud providers who need unified management. Hybrid cloud scenarios where Hetzner is one of several providers.

Talos Linux

Talos is an immutable, secure operating system built specifically for Kubernetes.

Considerations

More complex setup: Deploying Talos on Hetzner requires: - Manually creating network components - Setting up a NAT gateway VM - Generating Talos-specific configuration files - Bootstrapping nodes individually

Different operational model: Talos has no SSH access and is managed entirely via API. This provides enhanced security but requires adapting your workflows.

When Talos makes sense: Organizations prioritizing immutable infrastructure and maximum security. Teams comfortable with the Talos operational model.

Cluster API (CAPH)

Cluster API provides declarative, Kubernetes-style management of cluster infrastructure.

Considerations

Requires a management cluster: Like Claudie, you need an existing Kubernetes cluster (often a local kind cluster) to provision your workload cluster.

Steeper learning curve: Understanding Cluster API concepts and CAPH-specific resources takes time.

Kubernetes-native approach: If you're comfortable with Kubernetes operators and CRDs, this model may feel natural.

When CAPH makes sense: Organizations standardizing on Cluster API across multiple providers. Teams that want to manage infrastructure with kubectl and GitOps.

Real-World Considerations

Development and Testing

For quick iteration, hetzner-k3s excels. Creating and destroying clusters in minutes enables: - Ephemeral test environments - Rapid prototyping - Cost-effective experimentation (pay only for what you use)

Small to Medium Production

Most teams running 1-50 node clusters on Hetzner find hetzner-k3s sufficient. You get: - High availability with multi-location masters - Autoscaling for variable workloads - Zero recurring platform fees

Large Scale (100+ nodes)

hetzner-k3s has been tested with 500 nodes and is designed to scale beyond. Clusters over 100 nodes require some configuration changes—see the Recommendations page for setup details.

Multi-Cloud Requirements

If you need clusters across AWS, GCP, and Hetzner with unified management, consider Claudie or Cluster API. hetzner-k3s is designed specifically for Hetzner.

The Bottom Line

hetzner-k3s is designed for teams who want:

- Production-ready clusters in minutes, not hours
- Complete control over credentials and data
- No ongoing platform fees
- Minimal tooling complexity

It's not the right choice if you:

- Want someone else to handle all operations (consider managed services)
- Need multi-cloud standardization (consider Cluster API or Claudie)
- Already have extensive Terraform infrastructure (consider terraform-hcloud-kube-hetzner)

For most teams running Kubernetes on Hetzner Cloud, hetzner-k3s provides the best balance of speed, simplicity, and control.

Section:

https://vitobotta.github.io/hetzner-k3s/Contributing_and_support/

Contributing and Support

hetzner-k3s is an open source project, and contributions are welcome!

Getting Help

GitHub Issues

If you're running into issues with the tool, please open an issue. Include:

- Your configuration file (with sensitive values redacted)
- Full output with debug mode enabled (`DEBUG=true hetzner-k3s ...`)
- Your operating system and hetzner-k3s version
- Steps to reproduce the issue

GitHub Discussions

For general questions, ideas, or discussions, use GitHub Discussions. This is a good place for:

- Questions about best practices
- Feature suggestions
- Sharing how you're using hetzner-k3s

Documentation

Check the Troubleshooting page for solutions to common issues.

Contributing Code

Pull requests are welcome! Whether it's fixing a bug, improving documentation, or adding a feature.

Development Environment

hetzner-k3s is built using Crystal. You can develop using:

- **VS Code with Dev Containers** (recommended)
- **Docker Compose**
- **Local Crystal installation**

VS Code Setup

1. Install Visual Studio Code and the Dev Containers extension
2. Open the project in VS Code (`code .` in the repository root)
3. Click "Reopen in Container" when prompted
4. Wait for the container to build
5. Open a terminal inside the container

Note: If you can't find the Dev Containers extension, ensure you're using the official VS Code build (some extensions are disabled in Open Source builds).

Docker Compose Setup

If you prefer not to use VS Code:

```
# Build and start the development container
docker compose up -d

# Access the container
docker compose exec hetzner-k3s bash
```

Running the Tool

Inside the development container:

```
# Run without building
crystal run ./src/hetzner-k3s.cr -- create --config cluster_config.yaml

# Build a binary
crystal build ./src/hetzner-k3s.cr --static
```

The `--static` flag creates a statically linked binary that doesn't depend on external libraries.

Supporting the Project

If you or your company find this project useful, please consider becoming a sponsor. Your support helps:

- Fund ongoing development and maintenance
- Enable new features
- Keep the project actively maintained

Current Sponsors

Platinum: Alamos GmbH

Backers: @deubert-it, @jonasbadstuebner, @ricristian, @QuentinFAIDIDE

Consulting

Need help with hetzner-k3s, Kubernetes on Hetzner, or related infrastructure? The maintainer is available for consulting engagements.

Contact: vitobotta.com

Section:

https://vitobotta.github.io/hetzner-k3s/Creating_a_cluster/

Creating a Cluster

This page covers the full configuration reference for hetzner-k3s. For a quick start, see the Quick Start in the README or the Complete Tutorial.

Configuration File

hetzner-k3s uses a YAML configuration file. Below is a complete example with all options. Commented lines are optional:

```
---
hetzner_token: <your token>
cluster_name: test
kubeconfig_path: "./kubeconfig"
k3s_version: v1.32.0+k3s1

networking:
  ssh:
    port: 22
    use_agent: false # set to true if your key has a passphrase
    public_key_path: "~/.ssh/id_ed25519.pub"
    private_key_path: "~/.ssh/id_ed25519"
  allowed_networks:
    ssh:
      - 0.0.0.0/0
    api: # this will firewall port 6443 on the nodes
      - 0.0.0.0/0
    # OPTIONAL: define extra inbound/outbound firewall rules.
    # Each entry supports the following keys:
    #   description (string, optional)
    #   direction   (in | out, default: in)
    #   protocol    (tcp | udp | icmp | esp | gre, default: tcp)
    #   port        (single port "80", port range "30000-32767", or
"any") – only relevant for tcp/udp
    #   source_ips  (array of CIDR blocks) – required when direction is
in
    #   destination_ips (array of CIDR blocks) – required when direction
is out
    #
    # IMPORTANT: Outbound traffic is allowed by default (implicit allow-
all).
    # If you add **any** outbound rule (direction: out), Hetzner Cloud
switches
    # the outbound chain to an implicit **deny-all**; only traffic
matching your
```

```
    # outbound rules will be permitted. Define outbound rules carefully
to avoid
    # accidentally blocking required egress (DNS, updates, etc.).
    # NOTE: Hetzner Cloud Firewalls support **max 50 entries per
firewall**. The built-
    # in rules (SSH, ICMP, node-port ranges, etc.) use ~10 slots. If the
sum of the
    # default rules plus your custom ones exceeds 50, hetzner-k3s will
abort with
    # an error.
    # custom_firewall_rules:
    #   - description: "Allow HTTP from any IPv4"
    #     direction: in
    #     protocol: tcp
    #     port: 80
    #     source_ips:
    #       - 0.0.0.0/0
    #   - description: "UDP game servers (outbound)"
    #     direction: out
    #     protocol: udp
    #     port: 60000-60100
    #     destination_ips:
    #       - 203.0.113.0/24
  public_network:
    ipv4: true
    ipv6: true
    # hetzner_ips_query_server_url: https://.. # for large clusters, see
https://github.com/vitobotta/hetzner-
k3s/blob/main/docs/Recommendations.md
    # use_local_firewall: false # for large clusters, see
https://github.com/vitobotta/hetzner-
k3s/blob/main/docs/Recommendations.md
  private_network:
    enabled: true
    subnet: 10.0.0.0/16
    existing_network_name: ""
  cni:
    enabled: true
    encryption: false
    mode: flannel
    cilium:
      # Optional: specify a path to a custom values file for Cilium Helm
chart
      # When specified, this file will be used instead of the default
values
      # helm_values_path: "./cilium-values.yaml"
      # chart_version: "v1.17.2"

  # cluster_cidr: 10.244.0.0/16 # optional: a custom IPv4/IPv6 network
CIDR to use for pod IPs
  # service_cidr: 10.43.0.0/16 # optional: a custom IPv4/IPv6 network
CIDR to use for service IPs. Warning, if you change this, you should
also change cluster_dns!
  # cluster_dns: 10.43.0.10 # optional: IPv4 Cluster IP for coredns
service. Needs to be an address from the service_cidr range

datastore:
```

```
  mode: etcd # etcd (default) or external
  external_datastore_endpoint: postgres://....
#  etcd:
#    # etcd snapshot configuration (optional)
#    snapshot_retention: 24
#    snapshot_schedule_cron: "0 * * * *"
#
#    # S3 snapshot configuration (optional)
#    s3_enabled: false
#    s3_endpoint: "" # Can also be set with ETCD_S3_ENDPOINT environment
variable
#    s3_region: "" # Can also be set with ETCD_S3_REGION environment
variable
#    s3_bucket: "" # Can also be set with ETCD_S3_BUCKET environment
variable
#    s3_access_key: "" # Can also be set with ETCD_S3_ACCESS_KEY
environment variable
#    s3_secret_key: "" # Can also be set with ETCD_S3_SECRET_KEY
environment variable
#    s3_folder: ""
#    s3_force_path_style: false

schedule_workloads_on_masters: false # set to true to allow pods to be
scheduled on master nodes (useful for small clusters)

# image: rocky-9 # optional: default is ubuntu-24.04
# autoscaling_image: 103908130 # optional, defaults to the `image`
setting
# snapshot_os: microos # optional: specified the os type when using a
custom snapshot

masters_pool:
  instance_type: cpx22
  instance_count: 3 # for HA; you can also create a single master
cluster for dev and testing (not recommended for production)
  locations: # You can choose a single location for single master
clusters or if you prefer to have all masters in the same location. For
regional clusters (which are only available in the eu-central network
zone), each master needs to be placed in a separate location.
    - fsn1
    - hel1
    - nbg1

worker_node_pools:
- name: small-static
  instance_type: cpx22
  instance_count: 4
  location: hel1
  # image: debian-11
  # labels: # Kubernetes labels to apply to nodes in this pool (for node
selection in workloads)
  #   - key: purpose
  #     value: blah
  # taints: # Kubernetes taints to apply to nodes in this pool (to repel
pods unless they tolerate the taint)
  #   - key: something
  #     value: value1:NoSchedule
```

```
- name: medium-autoscaled
  instance_type: cpx32
  location: fsn1
  autoscaling:
    enabled: true
    min_instances: 0
    max_instances: 3

# addons:
#   csi_driver:
#     enabled: true   # Hetzner CSI driver (default true). Set to false
to skip installation.
#     manifest_url: "https://raw.githubusercontent.com/hetznercloud/csi-
driver/v2.18.3/deploy/kubernetes/hcloud-csi.yml"
#   traefik:
#     enabled: false  # built-in Traefik ingress controller. Disabled by
default.
#   servicelb:
#     enabled: false  # built-in ServiceLB. Disabled by default.
#   metrics_server:
#     enabled: false  # Kubernetes metrics-server addon. Disabled by
default.
#   cluster_autoscaler:
#     enabled: true # Cluster Autoscaler addon (default true). Set to
false to omit autoscaling.
#     manifest_url:
"https://raw.githubusercontent.com/kubernetes/autoscaler/master/cluster-
autoscaler/cloudprovider/hetzner/examples/cluster-autoscaler-run-on-
master.yaml"
#     container_image_tag: "v1.34.2"
#     scan_interval: "10s"                        # How often cluster is
reevaluated for scale up or down
#     scale_down_delay_after_add: "10m"           # How long after scale
up that scale down evaluation resumes
#     scale_down_delay_after_delete: "10s"        # How long after node
deletion that scale down evaluation resumes
#     scale_down_delay_after_failure: "3m"        # How long after scale
down failure that scale down evaluation resumes
#     max_node_provision_time: "15m"              # Maximum time CA
waits for node to be provisioned
#   cloud_controller_manager:
#     enabled: true   # Hetzner Cloud Controller Manager (default true).
Disabling stops automatic LB provisioning for Service objects.
#     manifest_url: "https://github.com/hetznercloud/hcloud-cloud-
controller-manager/releases/download/v1.28.0/ccm-networks.yaml"
#   system_upgrade_controller:
#     enabled: true   # System Upgrade Controller (default true). Set to
false to omit autoscaling.
#     deployment_manifest_url: "https://github.com/rancher/system-
upgrade-controller/releases/download/v0.18.0/system-upgrade-
controller.yaml"
#     crd_manifest_url: "https://github.com/rancher/system-upgrade-
controller/releases/download/v0.18.0/crd.yaml"
#   embedded_registry_mirror:
#     enabled: false # Enables fast p2p distribution of container images
between nodes for faster pod startup. Check if your k3s version is
```

```
compatible before enabling this option. You can find more information at
https://docs.k3s.io/installation/registry-mirror

protect_against_deletion: true # prevents accidental deletion of the
cluster with the "hetzner-k3s delete" command

create_load_balancer_for_the_kubernetes_api: false # creates a load
balancer for HA API access; note: Hetzner firewalls can't yet restrict
access to load balancers by IP

k3s_upgrade_concurrency: 1 # how many nodes to upgrade at the same
time; increase for faster upgrades in large clusters, but higher values
may impact availability

# additional_packages:
# - somepackage

# additional_pre_k3s_commands:
# - apt update
# - apt upgrade -y

# additional_post_k3s_commands:
# - apt autoremove -y
# For more advanced usage like resizing the root partition for use with
Rook Ceph, see [Resizing root partition with additional post k3s
commands](./Resizing_root_partition_with_post_create_commands.md)

# kube_api_server_args:
# - arg1
# - ...
# kube_scheduler_args:
# - arg1
# - ...
# kube_controller_manager_args:
# - arg1
# - ...
# kube_cloud_controller_manager_args:
# - arg1
# - ...
# kubelet_args:
# - arg1
# - ...
# kube_proxy_args:
# - arg1
# - ...
# api_server_hostname: k8s.example.com # optional: DNS for the k8s API
LoadBalancer. After the script has run, create a DNS record with the
address of the API LoadBalancer.
```

Most settings are straightforward and easy to understand. To see a list of available k3s releases, you can run the command `hetzner-k3s releases`.

If you prefer not to include the Hetzner token directly in the config file—perhaps for use with CI or to safely commit the config to a repository—you can use the `HCLOUD_TOKEN` environment variable instead. This variable takes precedence over the config file.

When setting `masters_pool.instance_count`, keep in mind that if you set it to 1, the tool will create a control plane that is not highly available. For production clusters, it's better to set this to a number greater than 1. To avoid split brain issues with etcd, this number should be odd, and 3 is the recommended value. Additionally, for production environments, it's a good idea to configure masters in different locations using the `masters_pool.locations` setting.

The `datastore` section configures how Kubernetes stores its cluster state. The default mode is `etcd`, which runs an embedded etcd cluster on your master nodes—this is the recommended option for most deployments. For very large clusters or special requirements, you can use `external` mode with an external datastore (etcd, PostgreSQL, or MySQL) by specifying the connection string in `external_datastore_endpoint`. The etcd mode also supports optional S3 backup configuration for disaster recovery.

You can define any number of worker node pools, either static or autoscaled, and create pools with nodes of different specifications to handle various workloads. Each pool can have optional `labels` and `taints`. Labels are key-value pairs that help you target specific nodes when scheduling workloads using node selectors or affinity rules. Taints prevent pods from being scheduled on certain nodes unless the pods explicitly tolerate the taint—useful for dedicating nodes to specific workloads or keeping certain nodes free for particular purposes.

Settings, such as `additional_packages`, `additional_pre_k3s_commands`, and `additional_post_k3s_commands`, can be specified at the root level of the configuration file or for each individual pool if different settings are needed. If these settings are configured at the pool level, they will override any settings defined at the root level.

- `additional_pre_k3s_commands`: Commands executed before k3s installation
- `additional_post_k3s_commands`: Commands executed after k3s is installed and configured

For an example of using `additional_post_k3s_commands` to resize the root partition for use with storage solutions like Rook Ceph, see Resizing root partition with additional post k3s commands.

The `addons` section controls which components hetzner-k3s installs automatically:

- **csi_driver**: The Hetzner CSI driver enables persistent volumes backed by Hetzner block storage. Enabled by default; disable if you're using alternative storage solutions like Rook Ceph or Longhorn.
- **cloud_controller_manager**: Integrates with Hetzner Cloud to provision load balancers automatically when you create Kubernetes Service objects of type LoadBalancer. Enabled by default.

- **system_upgrade_controller**: Enables zero-downtime rolling upgrades of k3s across your cluster. Enabled by default.
- **cluster_autoscaler**: Automatically scales worker node pools based on resource demands. Enabled by default when you have autoscaling pools defined.
- **traefik**: k3s's built-in ingress controller. Disabled by default; enable if you want a quick ingress solution without installing your own.
- **servicelb**: k3s's built-in load balancer for bare-metal environments. Disabled by default; typically not needed when using Hetzner's load balancers.
- **metrics_server**: Enables `kubectl top` commands for viewing resource usage. Disabled by default.
- **embedded_registry_mirror**: Enables peer-to-peer distribution of container images between nodes for faster pod startup. Disabled by default; check k3s version compatibility before enabling.

The `api_server_hostname` setting is useful when you enable `create_load_balancer_for_the_kubernetes_api`. After the cluster is created, you can point this DNS name to the API load balancer's address, giving you a stable hostname for accessing the Kubernetes API.

Currently, Hetzner Cloud offers six locations: two in Germany (`nbg1` in Nuremberg and `fsn1` in Falkenstein), one in Finland (`hel1` in Helsinki), two in the USA (`ash` in Ashburn, Virginia and `hil` in Hillsboro, Oregon), and one in Singapore (`sin`). Be aware that not all instance types are available in every location, so it's a good idea to check the Hetzner site and their status page for details.

To explore the available instance types and their specifications, you can either check them manually when adding an instance within a project or run the following command with your Hetzner token:

```
curl -H "Authorization: Bearer $API_TOKEN"
'https://api.hetzner.cloud/v1/server_types'
```

To create the cluster run:

```
hetzner-k3s create --config cluster_config.yaml | tee create.log
```

This process will take a few minutes, depending on how many master and worker nodes you have.

Disabling public IPs (IPv4 or IPv6 or both) on nodes

To improve security and save on IPv4 address costs, you can disable the public interface for all nodes by setting `enable_public_net_ipv4: false` and

`enable_public_net_ipv6: false`. These settings are global and will apply to all master and worker nodes. If you disable public IPs, make sure to run hetzner-k3s from a machine that has access to the same private network as the nodes, either directly or through a VPN.

Additional networking setup is required via cloud-init, so it's important that the machine you use to run hetzner-k3s has internet access and DNS configured correctly. Otherwise, the cluster creation process will get stuck after creating the nodes. For more details and instructions, you can refer to this discussion.

Using alternative OS images

By default, the image used for all nodes is `ubuntu-24.04`, but you can specify a different default image by using the root-level `image` config option. You can also set different images for different static node pools by using the `image` config option within each node pool. For example, if you have node pools with ARM instances, you can specify the correct OS image for ARM. To do this, set `image` to `103908130` with the specific image ID.

However, for autoscaling, there's a current limitation in the Cluster Autoscaler for Hetzner. You can't specify different images for each autoscaled pool yet. For now, if you want to use a different image for all autoscaling pools, you can set the `autoscaling_image` option to override the default `image` setting.

To see the list of available images, run the following:

```
export API_TOKEN=...

curl -H "Authorization: Bearer $API_TOKEN"
'https://api.hetzner.cloud/v1/images?per_page=100'
```

Besides the default OS images, you can also use a snapshot created from an existing instance. When using custom snapshots, make sure to specify the **ID** of the snapshot or image, not the description you assigned when creating the template instance.

I've tested snapshots with openSUSE MicroOS, but other options might work as well. You can easily create a MicroOS snapshot using this Terraform-based tool. The process only takes a few minutes. Once the snapshot is ready, you can use it with hetzner-k3s by setting the `image` configuration option to the **ID** of the snapshot and `snapshot_os` to `microos`.

Keeping a Project per Cluster

If you plan to create multiple clusters within the same project, refer to the section on Configuring Cluster-CIDR and Service-CIDR. Ensure that each cluster has its own unique Cluster-CIDR and Service-CIDR. Overlapping ranges will cause issues. However, I still recommend separating clusters into different projects. This makes it easier to clean up resources—if you want to delete a cluster, simply delete the entire project.

Configuring Cluster-CIDR and Service-CIDR

Cluster-CIDR and Service-CIDR define the IP ranges used for pods and services, respectively. In most cases, you won't need to change these values. However, advanced setups might require adjustments to avoid network conflicts.

Changing the Cluster-CIDR (Pod IP Range): To modify the Cluster-CIDR, uncomment or add the `cluster_cidr` option in your cluster configuration file and specify a valid CIDR notation for the network. Make sure this network is not a subnet of your private network.

Changing the Service-CIDR (Service IP Range): To adjust the Service-CIDR, uncomment or add the `service_cidr` option in your configuration file and provide a valid CIDR notation. Again, ensure this network is not a subnet of your private network. Also, uncomment the `cluster_dns` option and provide a single IP address from the `service_cidr` range. This sets the IP address for the coredns service.

Sizing the Networks: The networks you choose should have enough space for your expected number of pods and services. By default, `/16` networks are used. Select an appropriate size, as changing the CIDR later is not supported.

Autoscaler Configuration

The cluster autoscaler automatically manages the number of worker nodes in your cluster based on resource demands. When you enable autoscaling for a worker node pool, you can also configure various timing parameters to fine-tune its behavior.

Basic Autoscaling Configuration

```
worker_node_pools:
- name: autoscaled-pool
  instance_type: cpx32
  location: fsn1
  autoscaling:
    enabled: true
    min_instances: 1
    max_instances: 10
```

Advanced Timing Configuration

You can customize the autoscaler's behavior with these optional parameters at the root level of your configuration:

```
cluster_autoscaler:
  scan_interval: "2m"                      # How often cluster is
reevaluated for scale up or down
  scale_down_delay_after_add: "10m"        # How long after scale up
that scale down evaluation resumes
  scale_down_delay_after_delete: "10s"     # How long after node
deletion that scale down evaluation resumes
  scale_down_delay_after_failure: "15m"    # How long after scale down
failure that scale down evaluation resumes
  max_node_provision_time: "15m"           # Maximum time CA waits for
node to be provisioned

worker_node_pools:
- name: autoscaled-pool
  instance_type: cpx32
  location: fsn1
  autoscaling:
    enabled: true
    min_instances: 1
    max_instances: 10
```

Parameter Descriptions

- `scan_interval` : Controls how frequently the cluster autoscaler evaluates whether scaling is needed. Shorter intervals mean faster response to load changes but more API calls.
- *Default*: `10s`
- `scale_down_delay_after_add` : Prevents the autoscaler from immediately scaling down after adding nodes. This helps avoid thrashing when workloads are still starting up.
- *Default*: `10m`
- `scale_down_delay_after_delete` : Adds a delay before considering more scale-down operations after a node deletion. This ensures the cluster stabilizes before further changes.
- *Default*: `10s`
- `scale_down_delay_after_failure` : When a scale-down operation fails, this parameter controls how long to wait before attempting another scale-down.
- *Default*: `3m`
- `max_node_provision_time` : Sets the maximum time the autoscaler will wait for a new node to become ready. This is particularly useful for clusters with private networks where provisioning might take longer.

- *Default*: `15m`

These settings apply globally to all autoscaling worker node pools in your cluster.

Idempotency

The `create` command can be run multiple times with the same configuration without causing issues, as the process is idempotent. If the process gets stuck or encounters errors (e.g., due to Hetzner API unavailability or timeouts), you can stop the command and rerun it with the same configuration to continue where it left off. Note that the kubeconfig will be overwritten each time you rerun the command.

Limitations:

- Using a snapshot instead of a default image will take longer to create instances compared to regular images.
- The `networking.allowed_networks.api` setting specifies which networks can access the Kubernetes API. This works with both single-master and multi-master clusters, but only when `create_load_balancer_for_the_kubernetes_api` is disabled. If the API load balancer is enabled, Hetzner's firewalls do not yet support load balancers, so the API would be exposed to the public internet regardless of the allowed networks configuration.
- If you enable autoscaling for a nodepool, avoid changing this setting later, as it can cause issues with the autoscaler.
- Autoscaling is only supported with Ubuntu or other default images, not snapshots.
- If you already have SSH keys in your Hetzner project, it's best to use a different key for your cluster—unless there's already a key with the same name and fingerprint as the one in your config file. Hetzner doesn't allow two keys with the same fingerprint in one project. So if you've already added the key from your config but under a different name, Hetzner won't let you add it again. In that case, hetzner-k3s will skip creating the key and won't inject any SSH key into the cluster nodes. Without an SSH key, Hetzner sets up the nodes with password login instead. That means you'll get an email for each node with its root password. Managing several nodes this way can get tricky. To avoid this, just use a new keypair for your cluster—unless the project already has a key that matches both the name and fingerprint in your config.
- SSH keys with passphrases can only be used if you set `networking.ssh.use_ssh_agent` to `true` and use an SSH agent to access your key. For example, on macOS, you can start an agent like this:

```
eval "$(ssh-agent -s)"
ssh-add --apple-use-keychain ~/.ssh/<private key>
```

Section:

https://vitobotta.github.io/hetzner-k3s/Deleting_a_cluster/

Deleting a Cluster

Basic Deletion

To delete a cluster, you need to run the following command:

```
hetzner-k3s delete --config cluster_config.yaml
```

This command will remove all the resources in the Hetzner Cloud project that were created by `hetzner-k3s`.

Important Considerations

Protection Against Deletion

Additionally, to delete a cluster, you must ensure that `protect_against_deletion` is set to `false`. When you execute the `delete` command, you'll also need to enter the cluster's name to confirm the deletion. These steps are in place to avoid accidentally deleting a cluster you intended to keep.

Resources Not Automatically Deleted

Keep in mind that the following resources created by your applications will not be deleted automatically. You'll need to remove those manually:

- **Load Balancers**: Load balancers created by your applications (via Services of type LoadBalancer)
- **Persistent Volumes**: Persistent volumes and their underlying storage
- **Floating IPs**: Any floating IPs you've manually attached to instances
- **Snapshots**: Any snapshots created from instances

This behavior is by design to prevent accidental data loss. These resources might be improved in future updates.

Manual Cleanup Steps

Before Deleting the Cluster

1. **Backup Important Data**: Ensure you have backups of any important data stored in persistent volumes
2. **Export Application Configurations**: Save any Kubernetes manifests, Helm values, or configurations you might need later
3. **Note Load Balancer IPs**: If you have applications with public IPs, note them down as they might change if you recreate the cluster

After Deleting the Cluster

Manual Cleanup

You can easily delete any remaining resources using the **Hetzner Cloud Console**:

1. **Log in** to your Hetzner Cloud Console
2. **Navigate to your project**
3. **Delete remaining resources** from the left sidebar:
4. **Load Balancers** → Select and delete any application load balancers
5. **Volumes** → Select and delete any persistent volumes
6. **Floating IPs** → Select and delete any unused floating IPs
7. **Snapshots** → Select and delete any unnecessary snapshots

This visual approach is recommended as it's easier to identify which resources belong to your cluster and avoid accidental deletions.

Troubleshooting Deletion Issues

Cluster Still Protected

If you get an error about the cluster being protected:

1. **Check Configuration**: Ensure `protect_against_deletion: false` is set in your config file
2. **Verify Cluster Name**: Make sure you're entering the correct cluster name when prompted
3. **Check kubeconfig**: Sometimes the cluster name is read from the kubeconfig location

Resources Stuck in Deletion

If some resources are stuck and not being deleted:

1. **Check Hetzner Console**: Log in to the Hetzner Cloud Console to see the current state
2. **Wait and Retry**: Sometimes there's a delay in API updates, wait a few minutes and retry

Network Resources Not Deleted

If networks, firewall rules, or other network resources remain:

1. **Check Dependencies**: Make sure no instances are still using the network, load balancer or firewall
2. **Delete Manually**: Use the Hetzner Console or API to clean up remaining network resources

---

Alternative: Delete Entire Project

If your cluster is the only thing in the Hetzner Cloud project, you might find it easier to delete the entire project instead:

1. **Go to Hetzner Cloud Console**
2. **Navigate to your project**
3. **Click on "Settings"**
4. **Select "Delete Project"**

This will delete everything in the project, including any resources you might have forgotten about.

> **Warning**: This is irreversible! Only do this if you're certain you don't need anything in the project.

Best Practices

Planning for Deletion

When setting up your cluster, consider:

1. **Use Projects Wisely**: Consider creating separate projects for different clusters or environments

2. **Document Dependencies**: Keep track of external resources that depend on your cluster

Post-Deletion Checklist

After deleting your cluster, verify:

- [ ] No instances are running
- [ ] No load balancers are active
- [ ] No volumes are attached
- [ ] No floating IPs are allocated
- [ ] Network usage has stopped
- [ ] Billing reflects the changes

Cost Monitoring

Monitor your Hetzner Cloud billing dashboard for a few days after deletion to ensure:

- No unexpected charges appear
- All compute resources have been properly terminated
- Network and storage costs stop accumulating

If you see unexpected charges, check for orphaned resources that might need manual cleanup.

Section:

https://vitobotta.github.io/hetzner-k3s/Floating_IP_egress/

Floating IP Egress

This guide explains how to configure a dedicated egress IP address for all outbound traffic from your cluster. This is useful when external services need to allowlist your cluster's IP address, or when you need a consistent source IP for outgoing connections.

This setup uses Cilium's egress gateway feature with a Hetzner floating IP.

Enable Cilium Egress Gateway

In your cluster configuration, enable Cilium with the egress gateway feature:

```
networking:
  cni:
    enabled: true
    mode: cilium
    cilium_egress_gateway: true
```

Create a Dedicated Egress Node Pool

Add a worker node pool that will serve as the egress gateway. This node will route all outbound traffic through the floating IP:

```
worker_node_pools:
  - name: egress
    instance_type: cax21
    location: hel1
    instance_count: 1
    autoscaling:
      enabled: false
    labels:
      - key: node.kubernetes.io/role
        value: "egress"
    taints:
      - key: node.kubernetes.io/role
        value: egress:NoSchedule
```

The taint ensures that regular workloads are not scheduled on this node—it's dedicated to handling egress traffic.

Assign a Floating IP

1. Create a floating IP in the Hetzner Cloud Console (or via API/CLI) in the same location as your egress node
2. Assign the floating IP to the egress node
3. Configure the floating IP on the node's network interface (this may require additional cloud-init commands or manual configuration)

Apply the Egress Gateway Policy

Create a `CiliumEgressGatewayPolicy` to route outbound traffic through the egress node:

```yaml
apiVersion: cilium.io/v2
kind: CiliumEgressGatewayPolicy
metadata:
  name: egress-global
spec:
  selectors:
    - podSelector: {}

  destinationCIDRs:
    - "0.0.0.0/0"
  excludedCIDRs:
    - "10.0.0.0/8"

  egressGateway:
    nodeSelector:
      matchLabels:
        node.kubernetes.io/role: egress
    egressIP: YOUR_FLOATING_IP
```

Replace `YOUR_FLOATING_IP` with your actual floating IP address.

Apply the policy:

```
kubectl apply -f egress-policy.yaml
```

How It Works

- `selectors`: Matches all pods (empty `podSelector` means all pods in the cluster)
- `destinationCIDRs`: Routes all external traffic (`0.0.0.0/0`) through the egress gateway
- `excludedCIDRs`: Excludes internal/private network traffic (`10.0.0.0/8`) from being routed through the gateway, allowing pod-to-pod and pod-to-service

communication to work normally

- `egressGateway`: Specifies which node handles the egress traffic and what source IP to use

Once configured, all outbound traffic from your cluster to external destinations will originate from your floating IP address.

Section:

https://vitobotta.github.io/hetzner-k3s/Important_upgrade_notes/

Important Upgrade Notes

OpenSSH Upgrade Notice - Friday, August 1, 2025

Critical Information

Due to a recent OpenSSH upgrade made available for Ubuntu, there is a significant risk that cluster nodes created with a version of hetzner-k3s prior to 2.3.4 might become unreachable via SSH once OpenSSH gets upgraded and the nodes are rebooted.

The Problem

The OpenSSH upgrade changes systemd socket configuration behavior, which can cause SSH connectivity issues if the socket configuration file `/etc/systemd/system/ssh.socket.d/listen.conf` is not properly configured to handle IPv6 binding.

Solution for Reachable Nodes

If the nodes in your cluster are still reachable via SSH, you can fix this issue by running the following command:

```
hetzner-k3s run --config <your-config-file> --script fix-ssh.sh
```

This command will automatically fix the contents of `/etc/systemd/system/ssh.socket.d/listen.conf` to ensure SSH connectivity continues working after the OpenSSH server upgrade.

The script is available at the root of this project's repository and will: - Create a backup of the original configuration file with a timestamp - Properly configure the socket file to handle both IPv4 and IPv6 connections - Preserve all existing `ListenStream` configurations - Restart the SSH socket to apply changes

Workaround for Unreachable Nodes

If your nodes are no longer reachable via SSH due to OpenSSH already having been upgraded, there is a manual workaround available:

1. Run the `kube-shell` script from the project's repository (in the `bin` directory)

2. Specify the name of a node to fix as the first and only argument
3. This will open an SSH-like session on the node via `kubectl` using a temporary privileged pod
4. Within this session, manually modify `/etc/systemd/system/ssh.socket.d/listen.conf` to append the line:

```
BindIPv6Only=default
```

Important: This manual method must be performed for each node individually Exercise caution when modifying system configuration files.

Affected Versions

- **Fixed in**: hetzner-k3s 2.3.5 and later
- **Affected**: All versions prior to 2.3.5

Recommendation

We strongly recommend upgrading to hetzner-k3s 2.3.5 or later and running the fix script proactively before any OpenSSH upgrades occur to prevent any connectivity issues.

Additional Resources

For more information about SSH configuration and troubleshooting, please refer to the Troubleshooting documentation.

Section:

https://vitobotta.github.io/hetzner-k3s/Installation/

Installation

Get hetzner-k3s running on your system in under a minute.

Prerequisites

Before installing hetzner-k3s, you'll need:

Requirement	Description
Hetzner Cloud account	Sign up here if you don't have one
API token	Create one in Cloud Console → Security → API Tokens (read & write permissions)
SSH key pair	For accessing cluster nodes
kubectl	For interacting with your cluster (installation guide)
Helm	For installing applications (installation guide)

---

macOS

Homebrew (Recommended)

```
brew install vitobotta/tap/hetzner_k3s
```

Homebrew also works on Linux — see the Linux section below.

Binary Installation

If you prefer not to use Homebrew, install the required dependencies first:

- libevent
- bdw-gc
- libyaml
- pcre
- gmp

Apple Silicon:

```
wget https://github.com/vitobotta/hetzner-
k3s/releases/download/v2.4.5/hetzner-k3s-macos-arm64
chmod +x hetzner-k3s-macos-arm64
sudo mv hetzner-k3s-macos-arm64 /usr/local/bin/hetzner-k3s
```

Intel:

```
wget https://github.com/vitobotta/hetzner-
k3s/releases/download/v2.4.5/hetzner-k3s-macos-amd64
chmod +x hetzner-k3s-macos-amd64
sudo mv hetzner-k3s-macos-amd64 /usr/local/bin/hetzner-k3s
```

Linux

Homebrew (Recommended)

Homebrew works on Linux as well as macOS.

```
brew install vitobotta/tap/hetzner_k3s
```

amd64 (x86_64)

```
wget https://github.com/vitobotta/hetzner-
k3s/releases/download/v2.4.5/hetzner-k3s-linux-amd64
chmod +x hetzner-k3s-linux-amd64
sudo mv hetzner-k3s-linux-amd64 /usr/local/bin/hetzner-k3s
```

arm64 (ARM)

```
wget https://github.com/vitobotta/hetzner-
k3s/releases/download/v2.4.5/hetzner-k3s-linux-arm64
chmod +x hetzner-k3s-linux-arm64
sudo mv hetzner-k3s-linux-arm64 /usr/local/bin/hetzner-k3s
```

Fedora and Similar Distributions

Some distributions (like Fedora) may have OpenSSL compatibility issues. If you encounter errors, set these environment variables before running hetzner-k3s:

```
export OPENSSL_CONF=/dev/null
export OPENSSL_MODULES=/dev/null
```

For convenience, add a wrapper function to your `~/.bashrc` or `~/.zshrc`:

```
hetzner-k3s() {
    OPENSSL_CONF=/dev/null OPENSSL_MODULES=/dev/null command hetzner-
k3s "$@"
}
```

Windows

Use the Linux binary with WSL (Windows Subsystem for Linux).

After installing WSL, follow the Linux installation instructions above.

Verify Installation

Check that hetzner-k3s is installed correctly:

```
hetzner-k3s --version
```

You should see the version number displayed.

Next Steps

Now that hetzner-k3s is installed:

1. **Create your first cluster** — Configuration reference and detailed options
2. **Set up a complete stack** — Tutorial with ingress, TLS, and a sample application

---

Updating

To update to the latest version:

Homebrew:

```
brew upgrade vitobotta/tap/hetzner_k3s
```

Binary installations: Download and replace the binary using the same steps as the initial installation.

Check the releases page for the latest version and changelog.

Section:

https://vitobotta.github.io/hetzner-k3s/Load_balancers/

Load Balancers

Hetnzer-k3s automatically installs and configures the Hetzner Cloud Controller Manager, which enables you to create and manage Hetzner Load Balancers directly from Kubernetes using Services of type `LoadBalancer`.

Overview

When you create a Service of type `LoadBalancer` in your cluster, the Cloud Controller Manager will automatically:

1. Create a new Hetzner Load Balancer
2. Configure it with the specified settings and annotations
3. Set up health checks for your pods
4. Update the Service status with the Load Balancer's external IP

Basic Configuration

Essential Annotations

At a minimum, you'll need these two annotations:

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
  annotations:
    load-balancer.hetzner.cloud/location: nbg1  # Ensures the load
balancer is in the same network zone as your nodes
    load-balancer.hetzner.cloud/use-private-ip: "true"  # Routes
traffic between the load balancer and nodes through the private network
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

Recommended Annotations

While the above are essential, I also recommend adding these annotations for production use:

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
  annotations:
    # Basic configuration
    load-balancer.hetzner.cloud/location: nbg1
    load-balancer.hetzner.cloud/use-private-ip: "true"

    # Additional recommended settings
    load-balancer.hetzner.cloud/hostname: app.example.com  # Custom
hostname for the load balancer
    load-balancer.hetzner.cloud/name: my-app-lb           # Custom name
for the load balancer
    load-balancer.hetzner.cloud/http-redirect-https: 'false'  # Disable
HTTP to HTTPS redirect (handled by ingress)
    load-balancer.hetzner.cloud/uses-proxyprotocol: 'true'    # Enable
proxy protocol to preserve client IP
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

Advanced Configuration

Proxy Protocol and Client IP Preservation

The proxy protocol is important because it allows your ingress controller and applications to detect the real client IP address.

```
annotations:
  load-balancer.hetzner.cloud/uses-proxyprotocol: 'true'
  load-balancer.hetzner.cloud/hostname: app.example.com
```

Why use hostname with proxy protocol?

Enabling the proxy protocol can cause issues with cert-manager failing HTTP-01 challenges. To fix this, Hetzner and other providers recommend using a hostname instead of an IP for the load balancer. For more details, read this explanation.

Multiple Services and Ports

You can expose multiple ports and services through the same load balancer:

```
apiVersion: v1
kind: Service
metadata:
  name: multi-port-app
  annotations:
    load-balancer.hetzner.cloud/location: nbg1
    load-balancer.hetzner.cloud/use-private-ip: "true"
    load-balancer.hetzner.cloud/uses-proxyprotocol: 'true'
spec:
  type: LoadBalancer
  selector:
    app: multi-port-app
  ports:
    - protocol: TCP
      name: http
      port: 80
      targetPort: 8080
    - protocol: TCP
      name: https
      port: 443
      targetPort: 8443
```

Load Balancer Algorithm

You can specify the load balancing algorithm:

```
annotations:
  load-balancer.hetzner.cloud/algorithm: round_robin  # Options:
round_robin, least_connections
```

Health Checks

Customize health check behavior:

```
annotations:
  load-balancer.hetzner.cloud/health-check-interval: "15s"
  load-balancer.hetzner.cloud/health-check-timeout: "10s"
  load-balancer.hetzner.cloud/health-check-retries: "3"
```

Example: NGINX Ingress Controller

Here's a complete example for deploying NGINX Ingress Controller with a Load Balancer:

```yaml
---
# ingress-nginx-values.yaml
controller:
  kind: DaemonSet
  service:
    annotations:
      # Set the location (must match your node locations)
      load-balancer.hetzner.cloud/location: nbg1

      # Load balancer name
      load-balancer.hetzner.cloud/name: nginx-ingress-lb

      # Use private network for internal communication
      load-balancer.hetzner.cloud/use-private-ip: "true"

      # Enable proxy protocol to preserve client IPs
      load-balancer.hetzner.cloud/uses-proxyprotocol: 'true'

      # Set hostname for the load balancer (replace with your domain)
      load-balancer.hetzner.cloud/hostname: ingress.example.com

      # Disable automatic HTTP to HTTPS redirect (handled by ingress)
      load-balancer.hetzner.cloud/http-redirect-https: 'false'
```

Install with Helm:

```
# Add the Helm repository
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

# Install ingress-nginx with custom annotations
helm upgrade --install \
  ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace \
  -f ingress-nginx-values.yaml
```

Available Locations

Hetzner Cloud has data centers in these locations:

Location Code	City	Country
`nbg1`	Nuremberg	Germany
`fsn1`	Falkenstein	Germany
`hel1`	Helsinki	Finland

Location Code	City	Country
`ash`	Ashburn, VA	USA
`hil`	Hillsboro, OR	USA
`sin`	Singapore	Singapore

Make sure to choose a location where your nodes are deployed or across multiple locations if you have a distributed setup.

Complete List of Annotations

For a full list of available annotations and their descriptions, refer to the official documentation.

Common annotations include:

Annotation	Description	Default
`load-balancer.hetzner.cloud/location`	Location of the load balancer	Required
`load-balancer.hetzner.cloud/use-private-ip`	Use private network for node communication	`false`
`load-balancer.hetzner.cloud/hostname`	Custom hostname for the load balancer	Auto-generated
`load-balancer.hetzner.cloud/name`	Custom name for the load balancer	Service name
`load-balancer.hetzner.cloud/uses-proxyprotocol`	Enable proxy protocol	`false`

Annotation	Description	Default
`load-balancer.hetzner.cloud/algorithm`	Load balancing algorithm	`round_robin`
`load-balancer.hetzner.cloud/http-redirect-https`	Enable HTTP to HTTPS redirect	`false`
`load-balancer.hetzner.cloud/health-check-interval`	Health check interval	`15s`
`load-balancer.hetzner.cloud/health-check-timeout`	Health check timeout	`10s`
`load-balancer.hetzner.cloud/health-check-retries`	Health check retry count	`3`

Troubleshooting

Load Balancer Stuck in "Pending"

If your load balancer fails to get an external IP:

1. **Check Annotations**: Ensure all required annotations are set correctly
2. **Verify Location**: Make sure the specified location exists and has capacity
3. **Check Logs**: Examine the Cloud Controller Manager logs:

```
kubectl logs -n kube-system -l k8s-app=hcloud-cloud-controller-manager
```

4. **Check Service**: Verify the Service definition is correct:

```
kubectl describe service <service-name>
```

Health Check Failures

If health checks are failing:

1. **Check Pod Status**: Ensure pods are running and ready
2. **Verify Port Mapping**: Confirm the targetPort matches your application port
3. **Check Network Policies**: Ensure no network policies are blocking traffic
4. **Review Pod Logs**: Check application logs for errors

Connection Issues

If you can't connect through the load balancer:

1. **Check Security Groups**: Verify firewall rules allow traffic
2. **Test Direct Access**: Try accessing pods directly to isolate the issue
3. **Check DNS**: If using a hostname, ensure DNS resolves correctly
4. **Monitor Traffic**: Use `kubectl logs` and `kubectl describe` to trace traffic flow

Best Practices

1. **Use Private Networks**: Always set `use-private-ip: "true"` for better security and performance
2. **Enable Proxy Protocol**: Use `uses-proxyprotocol: 'true'` to preserve client IP addresses
3. **Choose Right Location**: Place load balancers close to your users for lower latency
4. **Monitor Health**: Regularly check load balancer health and metrics
5. **Use Meaningful Names**: Set custom names for easier identification in the Hetzner console
6. **Configure DNS**: Set up proper DNS records for your load balancer hostnames

Section:

https://vitobotta.github.io/hetzner-k3s/Maintenance/

Maintenance

Adding Nodes

To add one or more nodes to a node pool, simply update the instance count in the configuration file for that node pool and run the `create` command again.

Scaling Down a Node Pool

To reduce the size of a node pool:

1. Lower the instance count in the configuration file to ensure the extra nodes are not recreated in the future.
2. Drain and delete the additional nodes from Kubernetes. These are typically the last nodes when sorted alphabetically by name (`kubectl drain Node` followed by `kubectl delete node <name>`).
3. Remove the corresponding instances from the cloud console if the Cloud Controller Manager doesn't handle this automatically. Make sure you delete the correct ones!

Replacing a Problematic Node

1. Drain and delete the node from Kubernetes (`kubectl drain <name>` followed by `kubectl delete node <name>`).
2. Delete the correct instance from the cloud console.
3. Run the `create` command again. This will recreate the missing node and add it to the cluster.

Converting a Non-HA Cluster to HA

Converting a single-master, non-HA cluster to a multi-master HA cluster is straightforward. Increase the masters instance count and rerun the `create` command. This will set up a load balancer for the API server (if enabled) and update the kubeconfig to direct API requests through the load balancer or one of the master

contexts. For production clusters, it's also a good idea to place the masters in different locations (refer to this page for more details).

Upgrading to a New Version of k3s

Step 1: Initiate the Upgrade

For the first upgrade of your cluster, simply run the following command to update to a newer version of k3s:

```
hetzner-k3s upgrade --config cluster_config.yaml --new-k3s-version
v1.27.1-rc2+k3s1
```

Specify the new k3s version as an additional parameter, and the configuration file will be updated automatically during the upgrade. To view available k3s releases, run the command `hetzner-k3s releases`.

Note: For single-master clusters, the API server will be briefly unavailable during the control plane upgrade.

Step 2: Monitor the Upgrade Process

After running the upgrade command, you must monitor the upgrade jobs in the `system-upgrade` namespace to ensure all nodes are successfully upgraded:

```
# Watch the upgrade progress for all nodes
watch kubectl get nodes -owide

# Monitor upgrade jobs and plans
watch kubectl get jobs,pods -n system-upgrade

# Check upgrade plans status
kubectl get plan -n system-upgrade -o wide

# Check upgrade job logs
kubectl logs -n system-upgrade -f job/<upgrade-job-name>
```

You will see the masters upgrading one at a time, followed by the worker nodes. The upgrade process creates upgrade jobs in the `system-upgrade` namespace that handle the actual node upgrades.

Step 3: Verify Upgrade Completion

Before proceeding, ensure all upgrade jobs have completed successfully and all nodes are running the new k3s version:

```
# Check that all upgrade jobs are completed
kubectl get jobs -n system-upgrade

# Verify all nodes are ready and running the new version
kubectl get nodes -o wide

# Check for any failed or pending jobs
kubectl get jobs -n system-upgrade --field-selector status.failed=1
kubectl get jobs -n system-upgrade --field-selector status.active=1
```

✅ **Upgrade Completion Checklist:**

- [ ] All upgrade jobs in `system-upgrade` namespace have completed
- [ ] All nodes show `Ready` status
- [ ] All nodes display the new k3s version in `kubectl get nodes -owide`
- [ ] No active or failed upgrade jobs remain

Step 4: Run Create Command (CRITICAL)

Once all upgrade jobs have completed and all nodes have been successfully updated to the new k3s version, you MUST run the create command:

```
hetzner-k3s create --config cluster_config.yaml
```

Why This Step is Essential:

The `upgrade` command automatically updates the k3s version in your cluster configuration file, but this step is crucial because:

1. **Updates Masters Configuration**: Ensures that any new master nodes provisioned in the future will use the correct (new) k3s version instead of the previous version
2. **Updates Worker Node Templates**: Updates the worker node pool configurations to ensure new worker nodes are created with the upgraded k3s version
3. **Synchronizes Cluster State**: Ensures the actual cluster state matches the desired state defined in your configuration file
4. **Prevents Version Mismatch**: Without this step, new nodes added to the cluster would be created with the old k3s version and would need to be upgraded again by the system upgrade controller, causing unnecessary delays and potential issues

If you skip this step and add new nodes to the cluster later, they will first be created with the old k3s version and then need to be upgraded again, which is inefficient and can cause compatibility issues.

What to Do If the Upgrade Doesn't Go Smoothly

If the upgrade stalls or doesn't complete for all nodes:

1. Clean up existing upgrade plans and jobs, then restart the upgrade controller:

```
kubectl -n system-upgrade delete job --all
kubectl -n system-upgrade delete plan --all

kubectl label node --all plan.upgrade.cattle.io/k3s-server-
plan.upgrade.cattle.io/k3s-agent-

kubectl -n system-upgrade rollout restart deployment system-upgrade-
controller
kubectl -n system-upgrade rollout status deployment system-upgrade-
controller
```

You can also check the logs of the system upgrade controller's pod:

```
kubectl -n system-upgrade \
  logs -f $(kubectl -n system-upgrade get pod -l pod-template-hash -o
jsonpath="{.items[0].metadata.name}")
```

If the upgrade stalls after upgrading the masters but before upgrading the worker nodes, simply cleaning up resources might not be enough. In this case, run the following to mark the masters as upgraded and allow the upgrade to continue for the workers:

```
kubectl label node <master1> <master2> <master2>
plan.upgrade.cattle.io/k3s-server=upgraded
```

Once all the nodes have been upgraded, remember to re-run the `hetzner-k3s create` command. This way, new nodes will be created with the new version right away. If you don't, they will first be created with the old version and then upgraded by the system upgrade controller.

Upgrading the OS on Nodes

1. Consider adding a temporary node during the process if your cluster doesn't have enough spare capacity.
2. Drain one node.
3. Update the OS and reboot the node.
4. Uncordon the node.
5. Repeat for the next node.

To automate this process, you can install the Kubernetes Reboot Daemon ("Kured"). For Kured to work effectively, ensure the OS on your nodes has unattended upgrades enabled, at least for security updates. For example, if the image is Ubuntu, add this to the configuration file before running the `create` command:

```
additional_packages:
- unattended-upgrades
- update-notifier-common
additional_post_k3s_commands:
- sudo systemctl enable unattended-upgrades
- sudo systemctl start unattended-upgrades
```

Refer to the Kured documentation for additional configuration options, such as maintenance windows.

Section:

https://vitobotta.github.io/hetzner-k3s/Masters_in_different_locations/

Masters in Different Locations

To ensure maximum availability, you can set up a regional cluster by placing each master in a different European location. The first master will be in Falkenstein, the second in Helsinki, and the third in Nuremberg (listed alphabetically). This setup is only possible in network zones with multiple locations, and currently, the only such zone is `eu-central`, which includes these three European locations. For other regions, only zonal clusters are supported. Regional clusters are limited to 3 masters because we only have these three locations available.

To create a regional cluster, set the `instance_count` for the masters pool to 3 and specify the `locations` setting as `fsn1`, `hel1`, and `nbg1`.

Converting a Single Master or Zonal Cluster to a Regional One

If you already have a cluster with a single master or three masters in the same European location, converting it to a regional cluster is straightforward. Just follow these steps carefully and be patient. Note that this requires hetzner-k3s version 2.2.4 or higher.

Before you begin, make sure to back up all your applications and data! This is crucial. While the migration process is relatively simple, there is always some level of risk involved.

- Set the `instance_count` for the masters pool to 3 if your cluster currently has only one master.
- Update the `locations` setting for the masters pool to include `fns1`, `hel1`, and `nbg1` like this:

```
locations:
- fns1
- hel1
- nbg1
```

The locations are always processed in alphabetical order, regardless of how you list them in the `locations` property. This ensures consistency, especially when replacing a master due to node failure or other issues.

- If your cluster currently has a single master, run the `create` command with the updated configuration. This will create `master2` in Helsinki and `master3` in Nuremberg. Wait for the operation to complete and confirm that all three masters are in a ready state.
- If `master1` is not in Falkenstein (fns1):
- Drain `master1`.
- Delete `master1` using the command `kubectl delete node {cluster-name}-master1`.
- Remove the `master1` instance via the Hetzner Console or the `hcloud` utility.
- Run the `create` command again. This will recreate `master1` in Falkenstein.
- SSH into each master and run the following commands to ensure `master1` has joined the cluster correctly:

```
sudo apt-get update
sudo apt-get install etcd-client

export ETCDCTL_API=3
export ETCDCTL_ENDPOINTS=https://[REDACTED].1:2379
export ETCDCTL_CACERT=/var/lib/rancher/k3s/server/tls/etcd/server-
ca.crt
export ETCDCTL_CERT=/var/lib/rancher/k3s/server/tls/etcd/server-
client.crt
export ETCDCTL_KEY=/var/lib/rancher/k3s/server/tls/etcd/server-
client.key

etcdctl member list
```

The last command should display something like this if everything is working properly:

```
285ab4b980c2c8c, started, test-master2-d[REDACTED]af,
https://[REDACTED]:2380, https://[REDACTED]:2379, false
aad3fac89b68bfb7, started, test-master1-5e550de0,
https://[REDACTED]:2380, https://[REDACTED]:2379, false
c[REDACTED]e25aef34e8, started, test-master3-0ed051a3,
https://[REDACTED]:2380, https://[REDACTED]:2379, false
```

- If `master2` is not in Helsinki, follow the same steps as with `master1` but for `master2`. This will recreate `master2` in Helsinki.
- If `master3` is not in Nuremberg, repeat the process for `master3`. This will recreate `master3` in Nuremberg.

That's it! You now have a regional cluster, which ensures continued operation even if one of the Hetzner locations experiences a temporary failure. I also recommend enabling the `create_load_balancer_for_the_kubernetes_api` setting to `true` if you don't already have a load balancer for the Kubernetes API.

Performance Considerations

This feature has been frequently requested, but I delayed implementing it until I could thoroughly test the configuration. I was concerned about latency issues, as etcd is sensitive to delays, and I wanted to ensure that the latency between the German locations and Helsinki wouldn't cause problems.

It turns out that the default heartbeat interval for etcd is 100ms, and the latency between Helsinki and Falkenstein/Nuremberg is only 25-27ms. This means the total round-trip time (RTT) for the Raft consensus is around 60-70ms, which is well within etcd's acceptable limits. After running benchmarks, everything works smoothly! So, there's no need to adjust the etcd configuration for this setup.

Section:

https://vitobotta.github.io/hetzner-k3s/Private_clusters_with_public_network_int...

Private clusters with public network interface disabled

By default, network access to nodes in a cluster created with hetzner-k3s is limited to the networks listed in the configuration file. Some users might want to completely turn off the public interface on their nodes instead.

This page offers a reference configuration to help you disable the public interface. Keep in mind that some steps might vary depending on the operating system you choose for your nodes. The example configuration has been tested successfully with Debian 12, as it's a bit simpler to work with compared to other OSes.

Please note that this configuration is designed for new clusters only. I haven't tested if it works to convert an existing cluster to one with the public interface disabled.

When setting up a cluster with disabled public network interfaces, remember you'll need a NAT gateway to access the cluster from outside Hetzner Cloud. Without it, your nodes won't be able to connect to the internet, and hetzner-k3s won't be able to install k3s on those nodes.

Another important thing to consider is that with the public network interface disabled on all nodes, you can't run hetzner-k3s from a computer outside the cluster's private network. So, you'll need to run hetzner-k3s from a cloud instance within the private network. You could use the same instance you're using as your NAT gateway for this purpose too.

Prerequisite: NAT Gateway

First off, you need to set up a NAT gateway for your Hetzner Cloud network. Follow the instructions on this page.

The guide uses Debian as an example, so make sure to review the page and adjust any settings according to the operating system you choose for your cluster nodes and the NAT gateway instance.

The TL;DR is this:

- [ ] First, create a private network in the Hetzner project where your cluster will reside. Choose any subnet you like, but for reference purposes, let's assume it's

10.0.0.0/16.

- [ ] Next, set up a cloud instance to act as your NAT gateway. Ensure it has a public IP address and connects to the private network you just created.
- [ ] Then, add a route on your private network for `0.0.0.0/0`, directing it to the IP address of your NAT gateway instance, which you can select from a dropdown menu.
- [ ] Finally, on the NAT gateway instance itself, tweak `/etc/network/interfaces`. Add these lines or adjust existing ones for your private network interface:

```
auto enp7s0
iface enp7s0 inet dhcp
    post-up echo 1 > /proc/sys/net/ipv4/ip_forward
    post-up iptables -t nat -A POSTROUTING -s '10.0.0.0/16' -o enp7s0 -
j MASQUERADE
```

Replace `10.0.0.0/16` with your actual subnet if it's different. Also, make sure to use the correct name for your private network interface if `enp7s0` isn't right—find this with the `ifconfig` command.

- [ ] Lastly, restart your NAT gateway instance to apply these changes.

Cluster configuration

- [ ] Edit the configuration file for your cluster and set both `ipv4` and `ipv6` to `false`, plus reference the existing private network you have already created:

```
  public_network:
    ipv4: false
    ipv6: false
  private_network:
    enabled : true
    subnet: 10.0.0.0/16
    existing_network_name: "<name of your private network>"
```

Also configure the allowed networks:

```
  allowed_networks:
    ssh:
      - 0.0.0.0/0
    api:
      - 0.0.0.0/0
```

- [ ] Since you're setting up a private cluster, it makes sense to turn off the load balancer for the Kubernetes API. You can do this by setting `create_load_balancer_for_the_kubernetes_api` to `false`.

- [ ] Also, if you want to use an OS image other than the default (`ubuntu-24.04`), you can configure it accordingly. For example, if you prefer Debian 12, you can set it up like this:

```
image: debian-12
autoscaling_image: debian-12
```

- [ ] Next, you need to set up network configuration commands. These steps will ensure that the nodes in your clusters use the NAT gateway to access the Internet. You can use either `additional_pre_k3s_commands` (before k3s installation) or `additional_post_k3s_commands` (after k3s installation) depending on your needs.

For `ubuntu-24.04` :

```
additional_pre_k3s_commands:
- printf "
[Match]\nName=enp7s0\n\n[Network]\nDHCP=yes\nGateway=10.0.0.1\n" >
/etc/systemd/network/10-enp7s0.network
- printf "[Resolve]\nDNS=185.12.64.2 185.12.64.1" >
/etc/systemd/resolved.conf
- systemctl restart systemd-networkd
- systemctl restart systemd-resolved
```

For `debian-12` :

```
additional_pre_k3s_commands:
- apt update
- apt upgrade -y
- apt install ifupdown resolvconf -y
- apt autoremove -y hc-utils
- apt purge -y hc-utils
- echo "auto enp7s0" > /etc/network/interfaces.d/60-private
- echo "iface enp7s0 inet dhcp" >> /etc/network/interfaces.d/60-private
- echo "    post-up ip route add default via 10.0.0.1"  >>
/etc/network/interfaces.d/60-private
- echo "[Resolve]" > /etc/systemd/resolved.conf
- echo "DNS=1.1.1.1 1.0.0.1" >> /etc/systemd/resolved.conf
- ifdown enp7s0
- ifup enp7s0
- systemctl start resolvconf
- systemctl enable resolvconf
- echo "nameserver 1.1.1.1" >> /etc/resolvconf/resolv.conf.d/head
- echo "nameserver 1.0.0.1" >> /etc/resolvconf/resolv.conf.d/head
- resolvconf --enable-updates
- resolvconf -u
```

Note about Ubuntu vs Debian: The Debian configuration requires installing `ifupdown` and related packages, which cannot be done on Ubuntu without internet access. Ubuntu 24.04 uses systemd-networkd by default, making the configuration simpler and

more suitable for private cluster setups where internet access is only available after the NAT gateway configuration is complete.

Replace `enp7s0` with your network interface name, and `10.0.0.1` with the correct gateway IP address for your subnet. Note that this is not the IP address of the NAT gateway instance; it's simply the first IP in the range.

One important thing to remember: these simple commands work great if you're using the same type of instances, like all AMD instances, for both your master and worker node pools. We're referencing a specific private network interface name here.

If you plan on using different types of instances in your cluster, you'll need to tweak these commands to use a more flexible method for identifying the correct private interface on each node.

Creating the cluster

Run the `create` command with hetzner-k3s as usual, but use this updated configuration from an instance connected to the same private network. For example, you can use the NAT gateway instance if you don't want to create another one.

The nodes will be able to access the Internet through the NAT gateway. Therefore, hetzner-k3s should complete creating the cluster successfully.

Section:

https://vitobotta.github.io/hetzner-k3s/Recommendations/

Recommendations

This page provides best practices and recommendations for different cluster sizes and use cases with hetzner-k3s.

Small to Medium Clusters (1-50 nodes)

The default configuration works well for small to medium-sized clusters, providing a simple, reliable setup with minimal configuration required.

Key Considerations

- **Private Network**: Enabled by default for better security
- **CNI**: Flannel for simplicity or Cilium for advanced features
- **Storage**: `hcloud-volumes` for persistence
- **Load Balancers**: Hetzner Load Balancers for production workloads
- **High Availability**: 3 master nodes for production clusters

Recommended Configuration

```yaml
hetzner_token: <your token>
cluster_name: my-cluster
kubeconfig_path: "./kubeconfig"
k3s_version: v1.32.0+k3s1

networking:
  ssh:
    port: 22
    use_agent: false
    public_key_path: "~/.ssh/id_ed25519.pub"
    private_key_path: "~/.ssh/id_ed25519"
  allowed_networks:
    ssh:
      - 0.0.0.0/0
    api:
      - 10.0.0.0/16  # Restrict to private network
  public_network:
    ipv4: true
    ipv6: true
  private_network:
    enabled: true
```

```
    subnet: 10.0.0.0/16
  cni:
    enabled: true
    encryption: false
    mode: flannel

masters_pool:
  instance_type: cpx22
  instance_count: 3  # For HA
  locations:
    - nbg1

worker_node_pools:
- name: workers
  instance_type: cpx32
  instance_count: 3
  location: nbg1
  autoscaling:
    enabled: true
    min_instances: 1
    max_instances: 5

protect_against_deletion: true
create_load_balancer_for_the_kubernetes_api: true
```

Large Clusters (50+ nodes)

For larger clusters, the default setup has some limitations that need to be addressed.

Limitations of Default Setup

Hetzner's private networks, used in hetzner-k3s' default configuration, only support up to 100 nodes. If your cluster is going to grow beyond that, you need to disable the private network in your configuration, to use the public network instead (no worries, all traffic between nodes is automatically encrypted with Wireguard).

Large Cluster Architecture (Since v2.2.8)

Support for large clusters has significantly improved since version 2.2.8. The main changes include:

1. **Custom Firewall**: Instead of using Hetzner's firewall (which is slow to update), a custom firewall solution was implemented
2. **IP Query Server**: A simple container that checks the Hetzner API every 30 seconds to get the list of all node IPs
3. **Automatic Updates**: Firewall rules are automatically updated without manual intervention

Setting Up Large Clusters

Step 1: Set Up IP Query Server

The IP query server runs as a simple container. You can easily set it up on any Docker-enabled server using the `docker-compose.yml` file in the `ip-query-server` folder of this repository.

```
# docker-compose.yml
version: '3.8'
services:
  ip-query-server:
    build: ./ip-query-server
    ports:
      - "8080:80"
    environment:
      - HETZNER_TOKEN=your_token_here
  caddy:
    image: caddy:2
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
    depends_on:
      - ip-query-server
```

Replace `example.com` in the Caddyfile with your actual domain name and `mail@example.com` with your email address for Let's Encrypt certificates.

Step 2: Update Cluster Configuration

```
hetzner_token: <your token>
cluster_name: large-cluster
kubeconfig_path: "./kubeconfig"
k3s_version: v1.32.0+k3s1

networking:
  ssh:
    port: 22
    use_agent: true  # Recommended for large clusters
    public_key_path: "~/.ssh/id_ed25519.pub"
    private_key_path: "~/.ssh/id_ed25519"
  allowed_networks:
    ssh:
      - 0.0.0.0/0  # Required for public network access
    api:
      - 0.0.0.0/0  # Required when private network is disabled
  public_network:
    ipv4: true
    ipv6: true
    # Use custom IP query server for large clusters
    hetzner_ips_query_server_url: https://ip-query.example.com
    use_local_firewall: true  # Enable custom firewall
```

```
  private_network:
    enabled: false  # Disable private network for >100 nodes
  cni:
    enabled: true
    encryption: true  # Enable encryption for public network
    mode: cilium  # Better for large scale deployments

# Larger cluster CIDR ranges
cluster_cidr: 10.244.0.0/15  # Larger range for more pods
service_cidr: 10.96.0.0/16   # Larger range for more services
cluster_dns: 10.96.0.10

datastore:
  mode: etcd  # or external for very large clusters
  # external_datastore_endpoint: postgres://...

masters_pool:
  instance_type: cpx32
  instance_count: 3
  locations:
    - nbg1
    - hel1
    - fsn1

worker_node_pools:
- name: compute
  instance_type: cpx42
  location: nbg1
  autoscaling:
    enabled: true
    min_instances: 5
    max_instances: 50
- name: storage
  instance_type: cpx52
  location: hel1
  autoscaling:
    enabled: true
    min_instances: 3
    max_instances: 20

addons:
  embedded_registry_mirror:
    enabled: true  # Recommended for large clusters

protect_against_deletion: true
create_load_balancer_for_the_kubernetes_api: true
k3s_upgrade_concurrency: 2  # Can upgrade more nodes simultaneously
```

Additional Large Cluster Considerations

Network Configuration

- **CIDR Sizing**: Use larger cluster and service CIDR ranges to accommodate more pods and services

- **Encryption**: Enable CNI encryption when using public networks
- **Firewall**: The custom firewall automatically manages allowed IPs without opening ports to the public

High Availability Setup

For production large clusters, consider:

1. **Multiple IP Query Servers**: Set up 2-3 instances behind a load balancer for better availability
2. **External Datastore**: Use PostgreSQL instead of etcd for better scalability
3. **Distributed Master Nodes**: Place masters in different locations
4. **Multiple Node Pools**: Different instance types for different workloads

Cluster Sizing Guidelines

Development/Tiny Clusters (< 5 nodes)

```
masters_pool:
  instance_type: cpx11
  instance_count: 1  # Single master for testing
worker_node_pools:
- name: workers
  instance_type: cpx11
  instance_count: 1
```

Small Production Clusters (5-20 nodes)

```
masters_pool:
  instance_type: cpx22
  instance_count: 3  # HA masters
  locations:
    - fsn1
    - hel1
    - nbg1
worker_node_pools:
- name: workers
  instance_type: cpx32
  instance_count: 3
  autoscaling:
    enabled: true
    min_instances: 1
    max_instances: 5
```

Medium Production Clusters (20-50 nodes)

```
masters_pool:
  instance_type: cpx32
  instance_count: 3
  locations:
    - fsn1
    - hel1
    - nbg1
worker_node_pools:
- name: web
  instance_type: cpx32
  location: nbg1
  autoscaling:
    enabled: true
    min_instances: 3
    max_instances: 10
- name: backend
  instance_type: cpx42
  location: hel1
  autoscaling:
    enabled: true
    min_instances: 2
    max_instances: 8
```

Large Production Clusters (50-200+ nodes)

Use the large cluster configuration shown above with: - Multiple node pools for different workloads - Custom firewall and IP query server - Larger instance types for masters - External datastore if needed

Performance Optimization

Embedded Registry Mirror

In v2.0.0, there's a new option to enable the `embedded registry mirror` in k3s. You can find more details here. This feature uses Spegel to enable peer-to-peer distribution of container images across cluster nodes.

Benefits: - Faster pod startup times - Reduced external registry calls - Better reliability when external registries are inaccessible - Cost savings on egress bandwidth

Configuration:

```
embedded_registry_mirror:
  enabled: true
```

> **Note**: Ensure your k3s version supports this feature before enabling.

Storage Selection

Use `hcloud-volumes` for:

- Production databases where the app does not take care of replication already
- Persistent application data
- Content that must survive pod restarts
- Applications requiring high availability

Use `local-path` for:

- High-performance caching (Redis, Memcached)
- High-performance databases (Postgres, MySQL) where the app takes care of replication already
- Temporary file storage
- Applications that can tolerate data loss
- Maximum IOPS performance

CNI Selection

Flannel

- **Pros**: Simple, lightweight, good for small clusters
- **Cons**: Limited features, doesn't scale well to very large clusters
- **Best for**: Small to medium clusters, simplicity

Cilium

- **Pros**: Advanced features, better performance scales well
- **Cons**: More complex setup, higher resource usage
- **Best for**: Medium to large clusters, advanced networking needs

Security Recommendations

Network Security

1. **Restrict SSH and API Access**: Use CIDR restrictions in `allowed_networks.api` and `allowed_networks.ssh`
2. **Use Private Networks**: When possible, use private networks for cluster communication
3. **Monitor Network Traffic**: Implement network policies and monitoring

SSH Security

1. **Use SSH Keys**: hetzner-k3s configures nodes with SSH keys by default
2. **SSH Agent**: Enable `use_agent: true` for passphrase-protected keys
3. **Key Rotation**: Regularly rotate SSH keys if needed
4. **Access Logs**: Monitor SSH access logs

Cluster Security

1. **RBAC**: Implement proper role-based access control
2. **Network Policies**: Use Kubernetes network policies
3. **Pod Security**: Implement pod security standards
4. **Regular Updates**: Keep k3s and components updated

Cost Optimization

Instance Selection

- **Right-size Instances**: Start smaller and scale up as needed
- **Use Autoscaling**: Only pay for what you use

Storage Optimization

- **Clean Up Volumes**: Regularly delete unused volumes
- **Use Local Storage**: For temporary data where appropriate
- **Monitor Usage**: Set up monitoring to identify unused storage

Network Optimization

- **Use Private Networks**: Reduce egress costs
- **Optimize Images**: Use smaller container images
- **Registry Mirror**: Reduce registry egress costs

Monitoring and Observability

Essential Monitoring

1. **Node Resources**: CPU, memory, disk usage

2. **Cluster Health**: Node readiness, pod status
3. **Network Traffic**: Bandwidth usage, connection counts
4. **Storage Performance**: I/O operations, latency

Recommended Tools

- **Prometheus + Grafana**: For metrics and dashboards
- **Loki**: For log aggregation
- **Alertmanager**: For alerting
- **Node Exporter**: For node metrics

Section:

https://vitobotta.github.io/hetzner-k3s/Recommendations/#large-clusters-50-nodes

Recommendations

This page provides best practices and recommendations for different cluster sizes and use cases with hetzner-k3s.

Small to Medium Clusters (1-50 nodes)

The default configuration works well for small to medium-sized clusters, providing a simple, reliable setup with minimal configuration required.

Key Considerations

- **Private Network**: Enabled by default for better security
- **CNI**: Flannel for simplicity or Cilium for advanced features
- **Storage**: `hcloud-volumes` for persistence
- **Load Balancers**: Hetzner Load Balancers for production workloads
- **High Availability**: 3 master nodes for production clusters

Recommended Configuration

```yaml
hetzner_token: <your token>
cluster_name: my-cluster
kubeconfig_path: "./kubeconfig"
k3s_version: v1.32.0+k3s1

networking:
  ssh:
    port: 22
    use_agent: false
    public_key_path: "~/.ssh/id_ed25519.pub"
    private_key_path: "~/.ssh/id_ed25519"
  allowed_networks:
    ssh:
      - 0.0.0.0/0
    api:
      - 10.0.0.0/16  # Restrict to private network
  public_network:
    ipv4: true
    ipv6: true
  private_network:
    enabled: true
```

```
    subnet: 10.0.0.0/16
  cni:
    enabled: true
    encryption: false
    mode: flannel

masters_pool:
  instance_type: cpx22
  instance_count: 3  # For HA
  locations:
    - nbg1

worker_node_pools:
- name: workers
  instance_type: cpx32
  instance_count: 3
  location: nbg1
  autoscaling:
    enabled: true
    min_instances: 1
    max_instances: 5

protect_against_deletion: true
create_load_balancer_for_the_kubernetes_api: true
```

Large Clusters (50+ nodes)

For larger clusters, the default setup has some limitations that need to be addressed.

Limitations of Default Setup

Hetzner's private networks, used in hetzner-k3s' default configuration, only support up to 100 nodes. If your cluster is going to grow beyond that, you need to disable the private network in your configuration, to use the public network instead (no worries, all traffic between nodes is automatically encrypted with Wireguard).

Large Cluster Architecture (Since v2.2.8)

Support for large clusters has significantly improved since version 2.2.8. The main changes include:

1. **Custom Firewall**: Instead of using Hetzner's firewall (which is slow to update), a custom firewall solution was implemented
2. **IP Query Server**: A simple container that checks the Hetzner API every 30 seconds to get the list of all node IPs
3. **Automatic Updates**: Firewall rules are automatically updated without manual intervention

Setting Up Large Clusters

Step 1: Set Up IP Query Server

The IP query server runs as a simple container. You can easily set it up on any Docker-enabled server using the `docker-compose.yml` file in the `ip-query-server` folder of this repository.

```
# docker-compose.yml
version: '3.8'
services:
  ip-query-server:
    build: ./ip-query-server
    ports:
      - "8080:80"
    environment:
      - HETZNER_TOKEN=your_token_here
  caddy:
    image: caddy:2
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
    depends_on:
      - ip-query-server
```

Replace `example.com` in the Caddyfile with your actual domain name and `mail@example.com` with your email address for Let's Encrypt certificates.

Step 2: Update Cluster Configuration

```
hetzner_token: <your token>
cluster_name: large-cluster
kubeconfig_path: "./kubeconfig"
k3s_version: v1.32.0+k3s1

networking:
  ssh:
    port: 22
    use_agent: true  # Recommended for large clusters
    public_key_path: "~/.ssh/id_ed25519.pub"
    private_key_path: "~/.ssh/id_ed25519"
  allowed_networks:
    ssh:
      - 0.0.0.0/0  # Required for public network access
    api:
      - 0.0.0.0/0  # Required when private network is disabled
  public_network:
    ipv4: true
    ipv6: true
    # Use custom IP query server for large clusters
    hetzner_ips_query_server_url: https://ip-query.example.com
    use_local_firewall: true  # Enable custom firewall
```

```
  private_network:
    enabled: false  # Disable private network for >100 nodes
  cni:
    enabled: true
    encryption: true  # Enable encryption for public network
    mode: cilium  # Better for large scale deployments

# Larger cluster CIDR ranges
cluster_cidr: 10.244.0.0/15  # Larger range for more pods
service_cidr: 10.96.0.0/16   # Larger range for more services
cluster_dns: 10.96.0.10

datastore:
  mode: etcd  # or external for very large clusters
  # external_datastore_endpoint: postgres://...

masters_pool:
  instance_type: cpx32
  instance_count: 3
  locations:
    - nbg1
    - hel1
    - fsn1

worker_node_pools:
- name: compute
  instance_type: cpx42
  location: nbg1
  autoscaling:
    enabled: true
    min_instances: 5
    max_instances: 50
- name: storage
  instance_type: cpx52
  location: hel1
  autoscaling:
    enabled: true
    min_instances: 3
    max_instances: 20

addons:
  embedded_registry_mirror:
    enabled: true  # Recommended for large clusters

protect_against_deletion: true
create_load_balancer_for_the_kubernetes_api: true
k3s_upgrade_concurrency: 2  # Can upgrade more nodes simultaneously
```

Additional Large Cluster Considerations

Network Configuration

- **CIDR Sizing**: Use larger cluster and service CIDR ranges to accommodate more pods and services

- **Encryption**: Enable CNI encryption when using public networks
- **Firewall**: The custom firewall automatically manages allowed IPs without opening ports to the public

High Availability Setup

For production large clusters, consider:

1. **Multiple IP Query Servers**: Set up 2-3 instances behind a load balancer for better availability
2. **External Datastore**: Use PostgreSQL instead of etcd for better scalability
3. **Distributed Master Nodes**: Place masters in different locations
4. **Multiple Node Pools**: Different instance types for different workloads

Cluster Sizing Guidelines

Development/Tiny Clusters (< 5 nodes)

```
masters_pool:
  instance_type: cpx11
  instance_count: 1  # Single master for testing
worker_node_pools:
- name: workers
  instance_type: cpx11
  instance_count: 1
```

Small Production Clusters (5-20 nodes)

```
masters_pool:
  instance_type: cpx22
  instance_count: 3  # HA masters
  locations:
    - fsn1
    - hel1
    - nbg1
worker_node_pools:
- name: workers
  instance_type: cpx32
  instance_count: 3
  autoscaling:
    enabled: true
    min_instances: 1
    max_instances: 5
```

Medium Production Clusters (20-50 nodes)

```
masters_pool:
  instance_type: cpx32
  instance_count: 3
  locations:
    - fsn1
    - hel1
    - nbg1
worker_node_pools:
- name: web
  instance_type: cpx32
  location: nbg1
  autoscaling:
    enabled: true
    min_instances: 3
    max_instances: 10
- name: backend
  instance_type: cpx42
  location: hel1
  autoscaling:
    enabled: true
    min_instances: 2
    max_instances: 8
```

Large Production Clusters (50-200+ nodes)

Use the large cluster configuration shown above with: - Multiple node pools for different workloads - Custom firewall and IP query server - Larger instance types for masters - External datastore if needed

Performance Optimization

Embedded Registry Mirror

In v2.0.0, there's a new option to enable the `embedded registry mirror` in k3s. You can find more details here. This feature uses Spegel to enable peer-to-peer distribution of container images across cluster nodes.

Benefits: - Faster pod startup times - Reduced external registry calls - Better reliability when external registries are inaccessible - Cost savings on egress bandwidth

Configuration:

```
embedded_registry_mirror:
  enabled: true
```

> **Note**: Ensure your k3s version supports this feature before enabling.

Storage Selection

Use `hcloud-volumes` for:

- Production databases where the app does not take care of replication already
- Persistent application data
- Content that must survive pod restarts
- Applications requiring high availability

Use `local-path` for:

- High-performance caching (Redis, Memcached)
- High-performance databases (Postgres, MySQL) where the app takes care of replication already
- Temporary file storage
- Applications that can tolerate data loss
- Maximum IOPS performance

CNI Selection

Flannel

- **Pros**: Simple, lightweight, good for small clusters
- **Cons**: Limited features, doesn't scale well to very large clusters
- **Best for**: Small to medium clusters, simplicity

Cilium

- **Pros**: Advanced features, better performance scales well
- **Cons**: More complex setup, higher resource usage
- **Best for**: Medium to large clusters, advanced networking needs

Security Recommendations

Network Security

1. **Restrict SSH and API Access**: Use CIDR restrictions in `allowed_networks.api` and `allowed_networks.ssh`
2. **Use Private Networks**: When possible, use private networks for cluster communication
3. **Monitor Network Traffic**: Implement network policies and monitoring

SSH Security

1. **Use SSH Keys**: hetzner-k3s configures nodes with SSH keys by default
2. **SSH Agent**: Enable `use_agent: true` for passphrase-protected keys
3. **Key Rotation**: Regularly rotate SSH keys if needed
4. **Access Logs**: Monitor SSH access logs

Cluster Security

1. **RBAC**: Implement proper role-based access control
2. **Network Policies**: Use Kubernetes network policies
3. **Pod Security**: Implement pod security standards
4. **Regular Updates**: Keep k3s and components updated

Cost Optimization

Instance Selection

- **Right-size Instances**: Start smaller and scale up as needed
- **Use Autoscaling**: Only pay for what you use

Storage Optimization

- **Clean Up Volumes**: Regularly delete unused volumes
- **Use Local Storage**: For temporary data where appropriate
- **Monitor Usage**: Set up monitoring to identify unused storage

Network Optimization

- **Use Private Networks**: Reduce egress costs
- **Optimize Images**: Use smaller container images
- **Registry Mirror**: Reduce registry egress costs

Monitoring and Observability

Essential Monitoring

1. **Node Resources**: CPU, memory, disk usage

2. **Cluster Health**: Node readiness, pod status
3. **Network Traffic**: Bandwidth usage, connection counts
4. **Storage Performance**: I/O operations, latency

Recommended Tools

- **Prometheus + Grafana**: For metrics and dashboards
- **Loki**: For log aggregation
- **Alertmanager**: For alerting
- **Node Exporter**: For node metrics

Section:

https://vitobotta.github.io/hetzner-k3s/Resizing_root_partition_with_post_create...

Resizing root partition with additional post k3s commands

When deploying storage solutions like Rook Ceph, you may need to resize the root partition to free up disk space for use by the storage system. This guide shows you how to use the `additional_post_k3s_commands` setting to automate this process.

Overview

By default, Hetzner Cloud instances automatically expand the root partition to use the entire available disk space. To manually manage partitions (e.g., for Rook Ceph storage), you need to disable this automatic behavior and then use custom commands to resize partitions according to your needs.

The following commands will:

1. Expand the GPT partition table to use the entire disk
2. Resize the root partition to use 50% of the disk
3. Create a new partition using the remaining space
4. Expand the filesystem to use the entire root partition

Prerequisites

Disable automatic root partition growth by setting `grow_root_partition_automatically` to `false`. You can set this globally for all nodes or override it per node pool.

Global Setting

Apply to all nodes in the cluster:

```
grow_root_partition_automatically: false
```

Per Node Pool Override

Configure different partitioning strategies per node pool:

```
worker_node_pools:
- name: storage-workers
  instance_type: cpx32
  location: fsn1
  grow_root_partition_automatically: false  # Disable for storage nodes
  additional_post_k3s_commands:
  - [ sgdisk, -e, /dev/sda ]
  - [ partprobe ]
  - [ parted, -s, /dev/sda, mkpart, primary, ext4, "50%", "100%" ]
  - [ growpart, /dev/sda, "1" ]
  - [ resize2fs, /dev/sda1 ]

- name: regular-workers
  instance_type: cpx22
  location: hel1
  # Inherits global setting (or true if global is not set)
  # grow_root_partition_automatically: true  # automatic growth
```

How it works: - Global setting defaults to `true` (automatic growth) - Per-pool setting overrides global setting - If not specified per pool, inherits global setting - When `false`, creates `/etc/growroot-disabled` to prevent automatic growth

Partition Commands

Add these `additional_post_k3s_commands` to disable automatic growth and manually resize partitions:

```
additional_post_k3s_commands:
- [ sgdisk, -e, /dev/sda ]
- [ partprobe ]
- [ parted, -s, /dev/sda, mkpart, primary, ext4, "50%", "100%" ]
- [ growpart, /dev/sda, "1" ]
- [ resize2fs, /dev/sda1 ]
```

Command Breakdown

1. **`[ sgdisk, -e, /dev/sda ]`**
2. Expands the GPT partition table to use the entire disk space
3. **`[ partprobe ]`**
4. Notifies the kernel of partition table changes
5. **`[ parted, -s, /dev/sda, mkpart, primary, ext4, "50%", "100%" ]`**
6. Creates a new partition using the remaining 50% of disk space
7. Available for Rook Ceph or other storage solutions

8. `[ growpart, /dev/sda, "1" ]`
9. Resizes root partition (partition 1) to use 50% of the disk
10. `[ resize2fs, /dev/sda1 ]`
11. Expands the ext4 filesystem to use the entire root partition

Result

After running these commands:

- Root partition (`/dev/sda1`) uses 50% of disk space
- New partition available for storage solutions like Rook Ceph
- Filesystem expanded to use entire root partition

Important Notes

- **Device Names**: Commands assume root disk is `/dev/sda` and root partition is `/dev/sda1`. Adjust if needed.
- **Test First**: Test on a non-critical node before production use.
- **Backup Data**: These commands are destructive. Backup important data before applying.
- **Root Privileges**: Commands run as root, which is standard for `additional_post_k3s_commands`.

Example Configuration

Complete cluster configuration with disk resizing for storage nodes:

```
grow_root_partition_automatically: true  # Default for most nodes

masters_pool:
  instance_type: cpx22
  instance_count: 3
  locations: [fsn1, hel1, nbg1]
  # Inherits global: true (automatic growth)

worker_node_pools:
- name: storage-workers
  instance_type: cpx32
  instance_count: 4
  location: fsn1
  grow_root_partition_automatically: false  # Override: manual
partitioning
  additional_post_k3s_commands:
```

```
    - [ sgdisk, -e, /dev/sda ]
    - [ partprobe ]
    - [ parted, -s, /dev/sda, mkpart, primary, ext4, "50%", "100%" ]
    - [ growpart, /dev/sda, "1" ]
    - [ resize2fs, /dev/sda1 ]

  - name: regular-workers
    instance_type: cpx22
    instance_count: 2
    location: hel1
    # Inherits global: true (automatic growth)

# Other cluster settings...
```

This setup gives you flexibility: regular nodes use automatic growth while storage nodes use custom partitioning optimized for Rook Ceph or similar storage solutions.

Section:

https://vitobotta.github.io/hetzner-k3s/Run_command/

The 'run' Command

The `hetzner-k3s run` command allows you to execute a single command or an entire script on either all nodes in your cluster or on a specific instance. This is particularly useful for maintenance tasks, configuration updates, and automated operations across your cluster.

Command Overview

```
hetzner-k3s run --config <config-file> [options]
```

Required Parameters

- `--config`, `-c`: The path to your cluster configuration YAML file

Execution Modes

1. Running Commands

Execute a single command on all cluster nodes:

```
hetzner-k3s run --config cluster.yaml --command "sudo apt update &&
sudo apt upgrade -y"
```

Execute a single command on a specific instance:

```
hetzner-k3s run --config cluster.yaml --command "hostname" --instance
"worker-node-1"
```

2. Running Scripts

Execute a script file on all cluster nodes:

```
hetzner-k3s run --config cluster.yaml --script fix-ssh.sh
```

Execute a script file on a specific instance:

```
hetzner-k3s run --config cluster.yaml --script fix-ssh.sh --instance
"master-node-1"
```

Option Parameters

- `--command`: The shell command to execute
- `--script`: The path to a script file to execute
- `--instance`: The name of a specific instance to run the command/script on (if not specified, runs on all instances)

Note: You must specify exactly one of either `--command` or `--script`.

Examples

Example 1: Check system information on all nodes

```
hetzner-k3s run --config my-cluster.yaml --command "uname -a && df -h"
```

Example 2: Update packages on a specific worker node

```
hetzner-k3s run --config my-cluster.yaml --command "sudo apt update &&
sudo apt list --upgradable" --instance worker-1
```

Example 3: Run the SSH fix script on all nodes

```
hetzner-k3s run --config my-cluster.yaml --script fix-ssh.sh
```

Example 4: Run a custom maintenance script on master node only

```
hetzner-k3s run --config my-cluster.yaml --script maintenance.sh --
instance master-1
```

How It Works

Command Execution

When using `--command`, hetzner-k3s: 1. Connects to each instance via SSH 2. Executes the specified command directly 3. Captures and displays the output 4. Returns the command completion status

Script Execution

When using `--script`, hetzner-k3s: 1. Validates the script file exists and is readable 2. Uploads the script to `/tmp/<script-name>` on each instance 3. Makes the script executable 4. Executes the script 5. Captures and displays the output 6. Automatically cleans up by removing the uploaded script file

Parallel Execution

The `run` command executes operations in parallel across all instances, significantly reducing the time required for cluster-wide operations. Each instance's output is displayed separately for clarity.

User Confirmation

Before execution, the command displays: - A summary of instances that will be affected - The command or script to be executed - A confirmation prompt requiring you to type "continue" to proceed

Error Handling

The command handles various error scenarios:

- **SSH Connection Issues**: If SSH connection fails, the error is displayed and execution continues on other instances
- **Script File Not Found**: If the specified script file doesn't exist, the command exits with an error
- **Permission Issues**: If the script file is not readable, the command exits with an error
- **Instance Not Found**: If a specific instance name doesn't exist in the cluster, the command exits with an error

Output Format

Output is organized by instance, making it easy to identify which node produced which output:

```
Found 3 instances in the cluster
Command to execute: hostname

Nodes that will be affected:
  - master-1 (192.168.1.100)
```

```
  - worker-1 (192.168.1.101)
  - worker-2 (192.168.1.102)

Type 'continue' to execute this command on all nodes: continue

=== Instance: master-1 (192.168.1.100) ===
master-1
Command completed successfully

=== Instance: worker-1 (192.168.1.101) ===
worker-1
Command completed successfully

=== Instance: worker-2 (192.168.1.102) ===
worker-2
Command completed successfully
```

Security Considerations

- Commands and scripts are executed with the permissions of the SSH user
- Use `sudo` within commands/scripts when root privileges are required
- Scripts are uploaded to `/tmp/` and executed from there, then automatically cleaned up
- Ensure your script files have appropriate permissions and are secure

Use Cases

Maintenance Operations

- System updates: `--command "sudo apt update && sudo apt upgrade -y"`
- Log cleanup: `--command "sudo journalctl --vacuum-time=7d"`
- Service restarts: `--command "sudo systemctl restart docker"`

Configuration Management

- Apply configuration changes across all nodes
- Deploy configuration files using scripts
- Update system settings

Troubleshooting

- Check system status: `--command "systemctl status"`
- Examine logs: `--command "journalctl -u k3s-agent -n 50"`

- Verify network connectivity: `--command "ping -c 3 google.com"`

Security Updates

- Apply security patches cluster-wide
- Update SSH configurations (like the fix-ssh.sh script)
- Modify firewall rules

Tips and Best Practices

1. **Test on a single instance first**: Use `--instance` to test commands/scripts on one node before applying to all nodes
2. **Use idempotent operations**: Design commands/scripts to be safe to run multiple times
3. **Capture output**: For long-running operations, consider redirecting output to files
4. **Handle errors gracefully**: Include error handling in your scripts when appropriate
5. **Use absolute paths**: In scripts, prefer absolute paths to avoid path-related issues

Integration with Cluster Operations

The `run` command is particularly powerful when combined with other hetzner-k3s operations:

- Use after cluster creation to apply initial configurations
- Run pre-upgrade checks before upgrading cluster components
- Execute post-upgrade verification commands
- Apply security patches across the entire cluster efficiently

Section:

https://vitobotta.github.io/hetzner-k3s/Setting_up_a_cluster/

By TitanFighter

Setting Up a Cluster

This guide will walk you through creating a fully functional Kubernetes cluster on Hetzner Cloud using hetzner-k3s, complete with ingress controller and a sample application.

Prerequisites

Before starting, ensure you have:

1. **Hetzner Cloud Account** with project and API token
2. **SSH Key Pair** for accessing cluster nodes
3. **kubectl** installed on your local machine (installation guide)
4. **Helm** installed on your local machine (installation guide)
5. **hetzner-k3s** installed on your local machine (installation guide)

Instructions

Installation of a "hello-world" project

For testing, we'll use this "hello-world" app: hello-world app

1. Create a file called `hetzner-k3s_cluster_config.yaml` with the following configuration. This setup is for a Highly Available (HA) cluster with 3 master nodes and 3 worker nodes. You can use 1 master and 1 worker for testing:

```
hetzner_token: ...
cluster_name: hello-world
kubeconfig_path: "./kubeconfig"  # or /cluster/kubeconfig if you are
going to use Docker
k3s_version: v1.32.0+k3s1

networking:
  ssh:
    port: 22
    use_agent: false
    public_key_path: "~/.ssh/id_rsa.pub"
```

```
    private_key_path: "~/.ssh/id_rsa"
  allowed_networks:
    ssh:
      - 0.0.0.0/0
    api:
      - 0.0.0.0/0

masters_pool:
  instance_type: cpx22
  instance_count: 3
  locations:
    - fsn1
    - hel1
    - nbg1

worker_node_pools:
- name: small
  instance_type: cpx22
  instance_count: 4
  location: hel1
- name: big
  instance_type: cpx32
  location: fsn1
  autoscaling:
    enabled: true
    min_instances: 0
    max_instances: 3
```

For more details on all the available settings, refer to the full config example in Creating a cluster.

1. Create the cluster: `hetzner-k3s create --config hetzner-k3s_cluster_config.yaml`
2. `hetzner-k3s` automatically generates a `kubeconfig` file for the cluster in the directory where you run the tool. You can either copy this file to `~/.kube/config` if it's the only cluster or run `export KUBECONFIG=./kubeconfig` in the same directory to access the cluster. After this, you can interact with your cluster using `kubectl`.

TIP: If you don't want to run `kubectl apply ...` every time, you can store all your configuration files in a folder and then run `kubectl apply -f /path/to/configs/ -R`.

1. Create a file: `touch ingress-nginx-annotations.yaml`
2. Add annotations to the file: `nano ingress-nginx-annotations.yaml`

```
# INSTALLATION
# 1. Install Helm: https://helm.sh/docs/intro/install/
# 2. Add ingress-nginx Helm repo: helm repo add ingress-nginx
https://kubernetes.github.io/ingress-nginx
# 3. Update information of available charts: helm repo update
# 4. Install ingress-nginx:
# helm upgrade --install \
# ingress-nginx ingress-nginx/ingress-nginx \
```

```
# --set controller.ingressClassResource.default=true \ # Remove this
line if you don't want Nginx to be the default Ingress Controller
# -f ./ingress-nginx-annotations.yaml \
# --namespace ingress-nginx \
# --create-namespace

# LIST of all ANNOTATIONS: https://github.com/hetznercloud/hcloud-
cloud-controller-
manager/blob/master/internal/annotation/load_balancer.go

controller:
  kind: DaemonSet
  service:
    annotations:
      # Germany:
      # - nbg1 (Nuremberg)
      # - fsn1 (Falkenstein)
      # Finland:
      # - hel1 (Helsinki)
      # USA:
      # - ash (Ashburn, Virginia)
      # Without this, the load balancer won't be provisioned and will
stay in "pending" state.
      # You can check this state using "kubectl get svc -n ingress-
nginx"
      load-balancer.hetzner.cloud/location: nbg1

      # Name of the load balancer. This name will appear in your
Hetzner cloud console under "Your project -> Load Balancers".
      # NOTE: This is NOT the load balancer created automatically for
HA clusters. You need to specify a different name here to create a
separate load balancer for ingress Nginx.
      load-balancer.hetzner.cloud/name: WORKERS_LOAD_BALANCER_NAME

      # Ensures communication between the load balancer and cluster
nodes happens through the private network.
      load-balancer.hetzner.cloud/use-private-ip: "true"

      # [ START: Use these annotations if you care about seeing the
actual client IP ]
      # "uses-proxyprotocol" enables the proxy protocol on the load
balancer so that the ingress controller and applications can see the
real client IP.
      # "hostname" is needed if you use cert-manager (LetsEncrypt SSL
certificates). It fixes HTTP01 challenges for cert-manager
(https://cert-manager.io/docs/).
      # Check this link for more details:
https://github.com/compumike/hairpin-proxy
      # In short: the easiest fix provided by some providers (including
Hetzner) is to configure the load balancer to use a hostname instead of
an IP.
      load-balancer.hetzner.cloud/uses-proxyprotocol: 'true'

      # 1. "yourDomain.com" must be correctly configured in DNS to
point to the Nginx load balancer; otherwise, certificate provisioning
won't work.
      # 2. If you use multiple domains, specify any one.
```

```
      load-balancer.hetzner.cloud/hostname: yourDomain.com
      # [ END: Use these annotations if you care about seeing the
actual client IP ]

      load-balancer.hetzner.cloud/http-redirect-https: 'false'
```

- Replace `yourDomain.com` with your actual domain.
- Replace `WORKERS_LOAD_BALANCER_NAME` with a name of your choice.
- Add the ingress-nginx Helm repo: `helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx`
- Update the Helm repo: `helm repo update`
- Install ingress-nginx:

```
helm upgrade --install \
ingress-nginx ingress-nginx/ingress-nginx \
--set controller.ingressClassResource.default=true \
-f ./ingress-nginx-annotations.yaml \
--namespace ingress-nginx \
--create-namespace
```

The `--set controller.ingressClassResource.default=true` flag configures this as the default Ingress Class for your cluster. Without this, you'll need to specify an Ingress Class for every Ingress object you deploy, which can be tedious. If no default is set and you don't specify one, Nginx will return a 404 Not Found page because it won't "pick up" the Ingress.

TIP: To delete it: `helm uninstall ingress-nginx -n ingress-nginx`. Be careful, as this will delete the current Hetzner load balancer, and installing a new ingress controller may create a new load balancer with a different public IP.

1. After a few minutes, check that the "EXTERNAL-IP" column shows an IP instead of "pending": `kubectl get svc -n ingress-nginx`
2. The `load-balancer.hetzner.cloud/uses-proxyprotocol: "true"` annotation requires `use-proxy-protocol: "true"` for ingress-nginx. To set this up, create a file: `touch ingress-nginx-configmap.yaml`
3. Add the following content to the file: `nano ingress-nginx-configmap.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  # Do not change the name - this is required by the Nginx Ingress
Controller
  name: ingress-nginx-controller
  namespace: ingress-nginx
data:
  use-proxy-protocol: "true"
```

1. Apply the ConfigMap: `kubectl apply -f ./ingress-nginx-configmap.yaml`
2. Open your Hetzner cloud console, go to "Your project -> Load Balancers," and find the PUBLIC IP of the load balancer with the name you used in the `load-balancer.hetzner.cloud/name: WORKERS_LOAD_BALANCER_NAME` annotation. Copy or note this IP.
3. Download the hello-world app: `curl https://raw.githubusercontent.com/vitobotta/hetzner-k3s/refs/heads/main/sample-deployment.yaml --output hello-world.yaml`
4. Edit the file to add the annotation and set the hostname:

```
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world
  annotations:                          # <<<--- Add annotation
    kubernetes.io/ingress.class: nginx  # <<<--- Add annotation
spec:
  rules:
  - host: hello-world.IP_FROM_STEP_13.nip.io # <<<--- Replace
`IP_FROM_STEP_13` with the IP from step 13.
  ....
```

1. Install the hello-world app: `kubectl apply -f hello-world.yaml`
2. Open http://hello-world.IP_FROM_STEP_13.nip.io in your browser. You should see the Rancher "Hello World!" page. The `host.IP_FROM_STEP_13.nip.io` (the `.nip.io` part is key) is a quick way to test things without configuring DNS. A query to a hostname ending in `.nip.io` returns the IP address in the hostname itself. If you enabled the proxy protocol as shown earlier, your public IP address should appear in the `X-Forwarded-For` header, meaning the application can "see" it.
3. To connect your actual domain, follow these steps:
4. Assign the IP address from step 13 to your domain in your DNS settings.
5. Change `- host: hello-world.IP_FROM_STEP_13.nip.io` to `- host: yourDomain.com`.
6. Run `kubectl apply -f hello-world.yaml`.
7. Wait until DNS records are updated.

If you need LetsEncrypt

1. Add the LetsEncrypt Helm repo: `helm repo add jetstack https://charts.jetstack.io`
2. Update the Helm repo: `helm repo update`

3. Install the LetsEncrypt certificates issuer:

```
helm upgrade --install \
--namespace cert-manager \
--create-namespace \
--set crds.enabled=true \
cert-manager jetstack/cert-manager
```

1. Create a file called `lets-encrypt.yaml` with the following content:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
  namespace: cert-manager
spec:
  acme:
    email: [REDACTED]
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: letsencrypt-prod-account-key
    solvers:
    - http01:
        ingress:
          class: nginx
```

1. Apply the file: `kubectl apply -f ./lets-encrypt.yaml`

2. Edit `hello-world.yaml` and add the settings for TLS encryption:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"  # <<<--- Add
annotation
    kubernetes.io/tls-acme: "true"                      # <<<--- Add
annotation
spec:
  rules:
  - host: yourDomain.com  # <<<---- Your actual domain
  tls: # <<<---- Add this block
  - hosts:
    - yourDomain.com
    secretName: yourDomain.com-tls # <<<--- Add reference to secret
  ....
```

TIP: If you didn't configure Nginx as the default Ingress Class, you'll need to add the `spec.ingressClassName: nginx` annotation.

1. Apply the changes: `kubectl apply -f ./hello-world.yaml`

FAQs

1. Can I use MetalLB instead of Hetzner's Load Balancer?

Yes, you can use MetalLB with floating IPs in Hetzner Cloud, but I wouldn't recommend it. The setup with Hetzner's standard load balancers is much simpler. Plus, load balancers aren't significantly more expensive than floating IPs, so in my opinion, there's no real benefit to using MetalLB in this case.

2. How do I create and push Docker images to a repository, and how can Kubernetes work with these images? (GitLab example)

On the machine where you create the image:

- Start by logging in to the Docker registry: `docker login registry.gitlab.com`.
- Build the Docker image: `docker build -t registry.gitlab.com/COMPANY_NAME/REPO_NAME:IMAGE_NAME -f /some/path/to/Dockerfile .`.
- Push the image to the registry: `docker push registry.gitlab.com/COMPANY_NAME/REPO_NAME:IMAGE_NAME`.

On the machine running Kubernetes:

- Generate a secret to allow Kubernetes to access the images: `kubectl create secret docker-registry gitlabcreds --docker-server=https://registry.gitlab.com --docker-username=MYUSER --docker-password=MYPWD --docker-email=MYEMAIL -n NAMESPACE_OF_YOUR_APP -o yaml > docker-secret.yaml`.
- Apply the secret: `kubectl apply -f docker-secret.yaml -n NAMESPACE_OF_YOUR_APP`.

3. How can I check the resource usage of nodes or pods?

You need the metrics-server installed in your cluster. There are two ways to enable it:

1. **Via hetzner-k3s config** (recommended): Enable it in the `addons` section of your cluster configuration file, and hetzner-k3s will automatically enable it in k3s:

   ```
   addons:
     metrics_server:
       enabled: true
   ```

2. **Manual installation**: Install via manifest or Helm from the metrics-server repository.

After installation, you can use either `kubectl top nodes` or `kubectl top pods -A` to view resource usage.

4. What is Ingress?

There are two types of "ingress" to understand: `Ingress Controller` and `Ingress Resources`.

In the case of Nginx:

- The `Ingress Controller` is Nginx itself (defined as `kind: Ingress`), while `Ingress Resources` are services (defined as `kind: Service`).
- The `Ingress Controller` has various annotations (rules). You can use these annotations in `kind: Ingress` to make them "global" or in `kind: Service` to make them "local" (specific to that service).
- The `Ingress Controller` consists of a Pod and a Service. The Pod runs the Controller, which continuously monitors the /ingresses endpoint in your cluster's API server for updates to available `Ingress Resources`.

5. How can I configure autoscaling to automatically set up IP routes for new nodes to use a NAT server?

First, you'll need a NAT server, as described in this Hetzner community tutorial.

Then, use `additional_post_k3s_commands` to run commands after k3s installation:

```
additional_packages:
  - ifupdown
additional_post_k3s_commands:
  - apt update
  - apt upgrade -y
  - apt autoremove -y
  - ip route add default via [REDACTED]  # Replace this with your
gateway IP
```

You can also use `additional_pre_k3s_commands` to run commands before k3s installation if needed.

Useful Commands

```
kubectl get service [serviceName] -A or -n [nameSpace]
kubectl get ingress [ingressName] -A or -n [nameSpace]
kubectl get pod [podName] -A or -n [nameSpace]
kubectl get all -A
kubectl get events -A
```

```
helm ls -A
helm uninstall [name] -n [nameSpace]
kubectl -n ingress-nginx get svc
kubectl describe ingress -A
kubectl describe svc -n ingress-nginx
kubectl delete configmap nginx-config -n ingress-nginx
kubectl rollout restart deployment -n NAMESPACE_OF_YOUR_APP
kubectl get all -A` does not include "ingress", so use `kubectl get ing
-A
```

Useful Links

- kubectl Cheat Sheet
- A visual guide on troubleshooting Kubernetes deployments

Section:

https://vitobotta.github.io/hetzner-k3s/Storage/

Storage

hetzner-k3s provides integrated storage solutions for your Kubernetes workloads. The Hetzner CSI Driver is automatically installed during cluster creation, enabling seamless integration with Hetzner's block storage services. If you prefer not to use the driver, you can disable its installation by setting `addons.csi_driver.enabled` to `false` in the cluster configuration file. Keep in mind that the minimum size for a volume is 10 Gi.

Overview

Two storage classes are available:

1. `hcloud-volumes` (default): Uses Hetzner's block storage based on Ceph, providing replicated and highly available storage
2. `local-path` : Uses local node storage for maximum IOPS performance (disabled by default)

Hetzner Block Storage (hcloud-volumes)

Features

- **Replicated**: Based on Ceph, ensuring data is replicated across multiple disks
- **Highly Available**: Redundant storage with no single point of failure
- **Minimum Size**: 10Gi (smaller requests will be automatically rounded up)
- **Maximum Size**: 10Ti per volume
- **Dynamic Provisioning**: Volumes are automatically created and attached when needed

Basic Usage

Create a Persistent Volume Claim (PVC) using the default storage class:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-data-pvc
spec:
```

```
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

This will automatically provision a 10Gi Hetzner volume and attach it to the pod that uses this PVC.

Example: WordPress with Persistent Storage

```
---
# Persistent Volume Claim for WordPress
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: wordpress-pvc
  labels:
    app: wordpress
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
---
# Persistent Volume Claim for MySQL
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  labels:
    app: mysql
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
---
# MySQL Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  labels:
    app: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
```

```yaml
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: "rootpassword"
        - name: MYSQL_DATABASE
          value: "wordpress"
        - name: MYSQL_USER
          value: "wordpress"
        - name: MYSQL_PASSWORD
          value: "wordpress"
        ports:
        - containerPort: 3306
        volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql
      volumes:
      - name: mysql-storage
        persistentVolumeClaim:
          claimName: mysql-pvc
---
# WordPress Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
  labels:
    app: wordpress
spec:
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        env:
        - name: WORDPRESS_DB_HOST
          value: "mysql"
        - name: WORDPRESS_DB_USER
          value: "wordpress"
        - name: WORDPRESS_DB_PASSWORD
          value: "wordpress"
        - name: WORDPRESS_DB_NAME
          value: "wordpress"
        ports:
        - containerPort: 80
        volumeMounts:
        - name: wordpress-storage
          mountPath: /var/www/html
```

```
      volumes:
      - name: wordpress-storage
        persistentVolumeClaim:
          claimName: wordpress-pvc
```

Local Path Storage

Overview

The Local Path storage class uses the node's local disk storage directly, providing higher IOPS and lower latency compared to network-attached storage. This is ideal for:

- High-performance databases (Redis, MongoDB, PostgreSQL)
- Caching systems
- Temporary storage
- Applications requiring maximum storage performance

Enabling Local Path Storage

To enable the `local-path` storage class, add this to your cluster configuration:

```
addons:
  local_path_storage_class:
    enabled: true
```

Usage Example

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: redis-cache-pvc
spec:
  storageClassName: local-path
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  selector:
    matchLabels:
      app: redis
```

```
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:alpine
        ports:
        - containerPort: 6379
        volumeMounts:
        - name: redis-data
          mountPath: /data
      volumes:
      - name: redis-data
        persistentVolumeClaim:
          claimName: redis-cache-pvc
```

Important Considerations

Advantages of Local Path Storage: - **Higher Performance**: No network overhead, direct disk access - **Lower Latency**: Faster read/write operations - **Reduced Cost**: No additional costs for network storage

Limitations of Local Path Storage: - **Not Highly Available**: Data is tied to specific nodes - **No Replication**: Data loss occurs if node fails, so it works best when the application takes care of replication already - **Limited to Single Node**: Pod can only be scheduled on the node where data resides - **Manual Migration**: Data must be manually migrated when moving pods

Storage Class Comparison

Feature	hcloud-volumes	local-path
High Availability	✅ Yes	❌ No
Data Replication	✅ Yes	❌ No
Performance	Good (Network)	Excellent (Local)
Maximum Size	10Ti	Limited by node disk

Feature	hcloud-volumes	local-path
Cost	Volume pricing	Included in instance
Use Case	Persistent data	Caching, temporary data, high-performance apps
Pod Migration	✅ Easy	❌ Manual

Advanced Storage Features

Volume Expansion

You can expand existing volumes online without downtime:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-expandable-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

To expand the volume:

```
# Edit the PVC to increase the size
kubectl edit pvc my-expandable-pvc

# Change storage: 10Gi to storage: 20Gi
```

The CSI driver will automatically resize the filesystem if supported.

Storage Best Practices

1. Choose the Right Storage Type

- **Use `hcloud-volumes` for:**
- Production databases
- Persistent application data

- Content that must survive pod restarts
- Applications requiring high availability
- **Use `local-path` for:**
- Caching layers (Redis, Memcached) and databases (Postgres, MySQL)
- Temporary file storage
- High-performance computing workloads
- Applications that can tolerate data loss

2. Monitor Storage Usage

```
# Check PVC usage
kubectl get pvc -A
kubectl describe pvc <pvc-name>

# Check actual disk usage on nodes
kubectl get nodes -o wide
ssh root@<node-ip> 'df -h'
```

3. Set Resource Limits

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: monitored-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
    # Optional: Set resource limits (enforced by storage class)
    limits:
      storage: 20Gi
```

4. Use Storage Classes Appropriately

```
# Explicitly specify storage class
spec:
  storageClassName: hcloud-volumes  # or local-path
```

5. Implement Backup Strategies

- **Application-level Backups**: Implement regular backups using tools like velero, restic, or application-specific backup solutions

- **Off-server Backups**: Ensure critical data is backed up to external storage or cloud storage
- **Monitoring**: Set up alerts for storage usage and disk space

Troubleshooting Storage Issues

PVC Stuck in Pending

1. **Check Storage Class**:

```
kubectl get sc
```

2. **Verify PVC Definition**:

```
kubectl describe pvc <pvc-name>
```

3. **Check CSI Driver Status**:

```
kubectl get pods -n kube-system | grep csi
kubectl logs -n kube-system <csi-pod-name>
```

Volume Mount Failures

1. **Check Volume Attachment**:

```
kubectl get pv
kubectl describe pv <pv-name>
```

2. **Verify Pod Definition**:

```
kubectl describe pod <pod-name>
```

3. **Check Node Capacity**:

```
kubectl describe node <node-name>
```

Performance Issues

1. **Monitor I/O**:

```
kubectl exec -it <pod-name> -- iostat -x 1
```

2. **Check Storage Type**: Ensure you're using the right storage class for your workload
3. **Consider Local Storage**: For high-performance workloads, consider switching to `local-path`

Cost Optimization

hcloud-volumes Costs

- **Monthly Charge**: Based on volume size (€0.04/GB per month)

Optimization Strategies

1. **Right-size Volumes**: Start with smaller sizes and expand as needed
2. **Use Local Storage**: For temporary or high-performance data
3. **Monitor Usage**: Identify and reclaim unused storage

Monitoring Commands

```
# Check storage usage across all namespaces
kubectl get pvc -A --no-headers | awk '{print $4, $5}'

# List all storage classes
kubectl get sc

# Check CSI driver pods
kubectl get pods -n kube-system | grep -E '(csi|storage)'

# Check volume health
kubectl get pv -o custom-
columns=NAME:.metadata.name,STATUS:.status.phase,CAPACITY:.spec.capacity.s
CLASS:.spec.storageClassName
```

Section:

https://vitobotta.github.io/hetzner-k3s/Troubleshooting/

Troubleshooting

This page covers common issues and their solutions. If you don't find an answer here, check GitHub Issues or ask in GitHub Discussions.

Common Issues and Solutions

SSH Connection Problems

If the tool stops working after creating instances and you experience timeouts, the issue might be related to your SSH key. This can happen if you're using a key with a passphrase or an older key, as newer operating systems may no longer support certain encryption methods.

Solutions: 1. **Enable SSH Agent**: Set `networking.ssh.use_agent` to `true` in your configuration file. This lets the SSH agent manage the key.

For macOS:

```
eval "$(ssh-agent -s)"
ssh-add --apple-use-keychain ~/.ssh/<private key>
```

For Linux:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/<private key>
```

1. **Test SSH Manually**: Verify you can SSH to the instances manually:

   ```
   ssh -i ~/.ssh/your_private_key root@<server_ip>
   ```

2. **Check Key Permissions**: Ensure your private key has correct permissions:

   ```
   chmod 600 ~/.ssh/your_private_key
   ```

Enable Debug Mode

You can run `hetzner-k3s` with the `DEBUG` environment variable set to `true` for more detailed output:

```
DEBUG=true hetzner-k3s create --config cluster_config.yaml
```

This will provide more detailed output, which can help you identify the root of the problem.

Cluster Creation Fails after Node Creation

Symptoms: Instances are created but cluster setup fails.

Possible Causes: - Network connectivity issues between nodes - Firewall blocking communication - Hetzner API rate limits

Solutions: 1. **Check Network Connectivity**: Verify nodes can communicate with each other 2. **Review Firewall Rules**: Ensure necessary ports are open 3. **Wait and Retry**: If it's a rate limit issue, wait a few minutes and retry 4. **Check Network Configuration**: See section below for IPv4/IPv6 configuration issues

IPv4 Disabled with IPv6 Only Configuration

Symptoms: Cluster creation hangs after nodes are created. SSH connection times out when trying to connect to a private IP address.

Note: The tool currently does not support IPv6-only public network configuration. When you disable IPv4 (`public_network.ipv4: false`), you must run `hetzner-k3s` from a machine that has access to the same private network, either directly or through a VPN. Otherwise, the tool will attempt to use the private IP addresses for SSH connections and fail.

Load Balancer Issues

Symptoms: Load balancer stuck in "pending" state

Solutions: 1. **Check Annotations**: Ensure proper annotations are set on your services 2. **Verify Location**: Make sure the load balancer location matches your node locations 3. **Check DNS Configuration**: If using hostname annotation, ensure DNS is properly configured

Node Not Ready

Symptoms: Nodes show up as `NotReady` status

Solutions: 1. **Check Node Status**:

```
kubectl describe node <node-name>
kubectl get nodes -o wide
```

1. **Check Kubelet**:

```
ssh -i ~/.ssh/your_private_key root@<node-ip>
systemctl status k3s-agent  # for workers
systemctl status k3s-server  # for masters
journalctl -u k3s-agent -f
```

2. **Restart K3s**:

```
ssh -i ~/.ssh/your_private_key root@<node-ip>
systemctl restart k3s-agent  # or k3s-server
```

Pod Stuck in Pending State

Symptoms: Pods remain in `Pending` state indefinitely

Solutions: 1. **Check Resource Availability**:

```
kubectl describe pod <pod-name> -n <namespace>
```

Look for events indicating insufficient resources.

1. **Add More Nodes**: If nodes are at capacity, either scale up existing node pools or add new nodes
2. **Check Taints and Tolerations**: Ensure pods have tolerations for any node taints

Storage Issues

Symptoms: PVCs stuck in `Pending` state, pods can't mount volumes

Solutions: 1. **Check Storage Classes**:

```
kubectl get sc
```

1. **Describe PVC**:

```
kubectl describe pvc <pvc-name> -n <namespace>
```

2. **Check CSI Driver**:

```
kubectl get pods -n kube-system | grep csi
```

Network Plugin Issues

Symptoms: Pods can't communicate with each other, DNS resolution fails

Solutions: 1. **Check CNI Pods**:

```
kubectl get pods -n kube-system | grep -E '(flannel|cilium)'
```

1. **Restart CNI**: Restart the relevant CNI pods

Upgrade Issues

Symptoms: Cluster upgrade process gets stuck

Solutions: 1. **Clean up Upgrade Resources**:

```
kubectl -n system-upgrade delete job --all
kubectl -n system-upgrade delete plan --all
```

1. **Remove Labels**:

```
kubectl label node --all plan.upgrade.cattle.io/k3s-server-
plan.upgrade.cattle.io/k3s-agent-
```

2. **Restart Upgrade Controller**:

```
kubectl -n system-upgrade rollout restart deployment system-upgrade-
controller
```

Getting Help

If you're still experiencing issues after trying these solutions:

1. **Check GitHub Issues**: Search existing issues at github.com/vitobotta/hetzner-k3s/issues
2. **Create New Issue**: If your issue hasn't been reported, create a new issue with:
3. Your configuration file (redacted)
4. Full debug output (`DEBUG=true hetzner-k3s ...`)

5. Operating system and Hetzner-k3s version
6. Steps to reproduce the issue
7. **GitHub Discussions**: For general questions and discussions, use GitHub Discussions

Useful Commands for Troubleshooting

```
# Check cluster status
kubectl cluster-info
kubectl get nodes
kubectl get pods -A

# Check resource usage
kubectl top nodes
kubectl top pods -A

# Check events
kubectl get events -A --sort-by='.metadata.creationTimestamp'

# Check specific pod details
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>

# Check node details
kubectl describe node <node-name>

# Check network connectivity
kubectl run test-pod --image=busybox -- sleep 3600
kubectl exec -it test-pod -- nslookup kubernetes.default
kubectl exec -it test-pod -- ping <other-pod-ip>
```

Section:

https://vitobotta.github.io/hetzner-k3s/Upgrading_a_cluster_from_1x_to_2x/

Upgrading a cluster created with hetzner-k3s v1.x to v2.x

The v1 version of hetzner-k3s is quite old and hasn't been supported for some time. I understand that many haven't upgraded to v2 because, until now, there wasn't a simple process to do this.

The good news is that the migration is now possible and straightforward, as long as you follow these instructions **very carefully** and take your time. This upgrade also allows you to replace deprecated instance types with newer ones. Note that this migration requires hetzner-k3s v2.2.4 or higher. Using the very latest version is recommended.

Prerequisites

- [ ] I suggest installing the hcloud utility. It will make it easier and faster to delete old master nodes.

Upgrading configuration and first steps

- [ ] **Backup apps and data** – As with any migration, there's some risk involved, so it's better to be prepared in case things don't go as planned.
- [ ] **Backup kubeconfig and the old config file**
- [ ] Uninstall the System Upgrade Controller
- [ ] Create a resolv file on existing nodes. You can do this manually or automate it using the `hcloud` CLI:

```
hcloud server list | awk '{print $4}' | tail -n +2 | while read ip;
do
  echo "Setting DNS for ${ip}"
  ssh -n root@${ip} "echo nameserver 8.8.8.8 | tee /etc/k8s-
resolv.conf"
  ssh -n root@${ip} "cat /etc/k8s-resolv.conf"
done
```

- [ ] Convert the config file to the new format. You can find guidance here for the initial v2.0.0 release. Make sure you read all the following release notes to apply any

other changes to the config format introduced by newer versions of the tool, until the very latest release.

- [ ] Remove or comment out empty node pools from the config file.
- [ ] Set `embedded_registry_mirror.enabled` to `false` if necessary, depending on the current version of k3s (refer to this documentation).
- [ ] Add `legacy_instance_type` to **ALL** node pools, including both masters and workers. Set it to the current instance type for each node pool, even if it's now deprecated. **This step is critical for the migration**.
- [ ] Set `instance` type for all the node pools to supported instance types, if your current ones have been deprecated by Hetzner. This is important to replace the existing nodes with new ones based on supported instance types.
- [ ] Run the `create` command **using the latest version of hetzner-k3s and the new config file**.
- [ ] Wait for all CSI pods in `kube-system` to restart, and **make sure everything is running correctly**.

Rotating control plane instances with the new instance type

Replace one master at a time (unless your cluster has a load balancer for the Kubernetes API, switch to another master's kube context before replacing `master1`):

- [ ] Drain the master first, then delete it both from the cluster using both kubectl and from the Hetzner console (or the `hcloud` CLI) to delete the actual instance.
- [ ] Rerun the `create` command to recreate the master with the new instance type. Wait for it to join the control plane and reach the "ready" status.
- [ ] SSH into each master and verify that the etcd members are updated and in sync:

```
sudo apt-get update
sudo apt-get install etcd-client

export ETCDCTL_API=3
export ETCDCTL_ENDPOINTS=https://127.0.0.1:2379
export ETCDCTL_CACERT=/var/lib/rancher/k3s/server/tls/etcd/server-
ca.crt
export ETCDCTL_CERT=/var/lib/rancher/k3s/server/tls/etcd/server-
client.crt
export ETCDCTL_KEY=/var/lib/rancher/k3s/server/tls/etcd/server-
client.key

etcdctl member list
```

Repeat the steps above carefully for each master. Once all three masters have been replaced:

- [ ] Rerun the `create` command once or twice more to ensure the configuration is stable and the masters no longer restart.
- [ ] Debug DNS resolution. If there are issues, restart the agents for DNS resolution with this command, then restart CoreDNS:

```
hcloud server list | grep worker | awk '{print $4}'| while read ip;
do
  echo "${ip}"
  ssh -n root@${ip} "systemctl restart k3s-agent"
  sleep 10
done
```

- [ ] Address any issues with your workloads before proceeding to rotate the worker nodes.

Rotating a worker node pool

- [ ] Increase the node count for the pool by 1.
- [ ] Run the `create` command to create the extra node needed during the pool rotation.

Replace one worker node at a time (except for the last one you just added):

- [ ] Drain a node.
- [ ] Delete the drained node using both kubectl and the Hetzner console (or the `hcloud` CLI).
- [ ] Rerun the `create` command to recreate the deleted node.
- [ ] Verify everything is working as expected before moving on to the next node in the pool.

Once all the existing nodes in the pool have been rotated:

- [ ] Drain the very last node in the pool (the one you added earlier).
- [ ] Verify everything is functioning correctly.
- [ ] Delete the last node using both kubectl and the Hetzner console (or the `hcloud` CLI).
- [ ] Update the `instance_count` for the node pool by reducing it by 1.
- [ ] Proceed with the next node pool.

Finalizing

- [ ] Remove the `legacy_instance_type` setting from both master and worker node pools.
- [ ] Rerun the `create` command once more to double-check everything.
- [ ] Optionally, convert the currently zonal cluster to a regional one with masters in different locations (see this guide).